

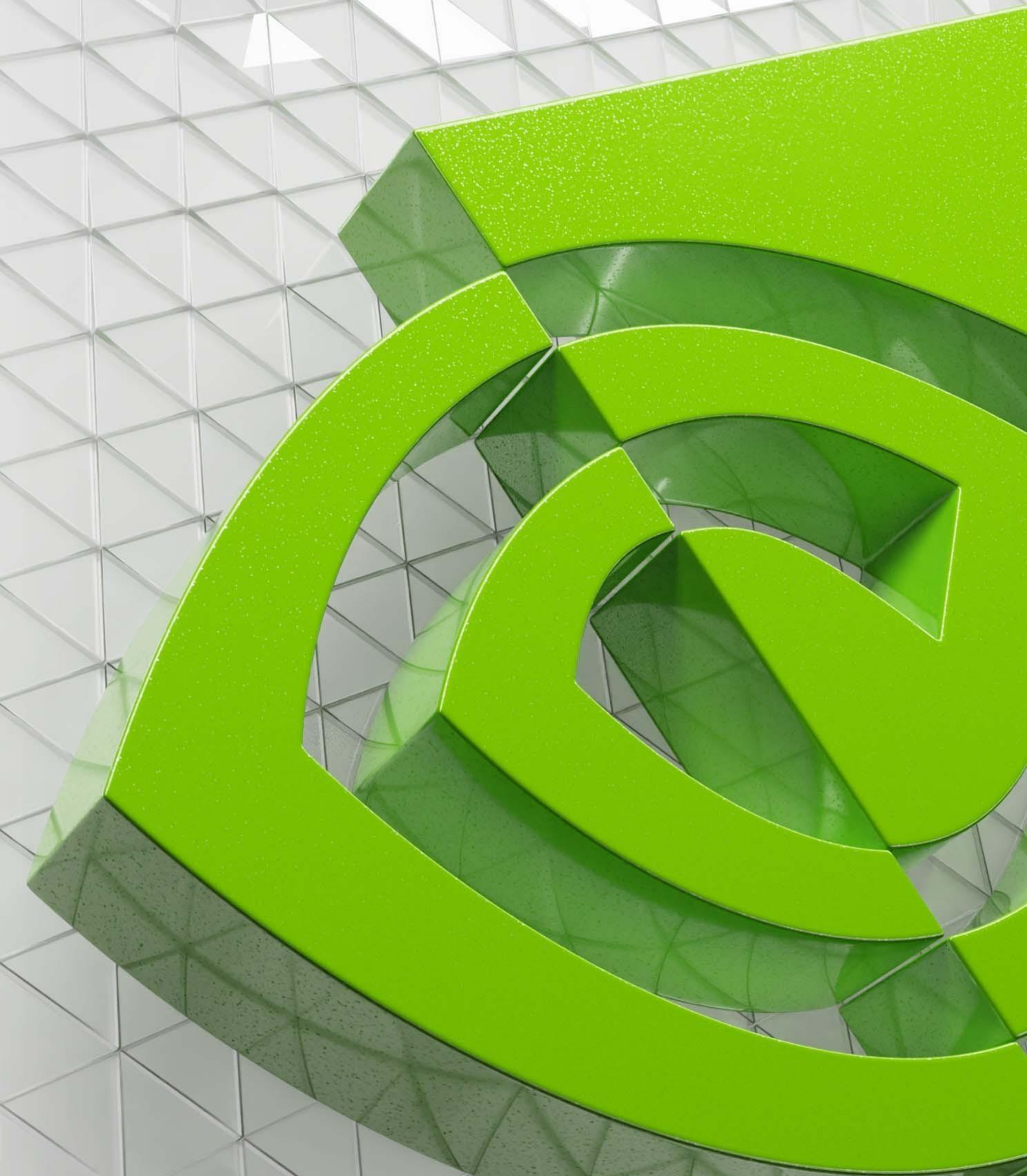


VSS 3.2.0 Technical Training



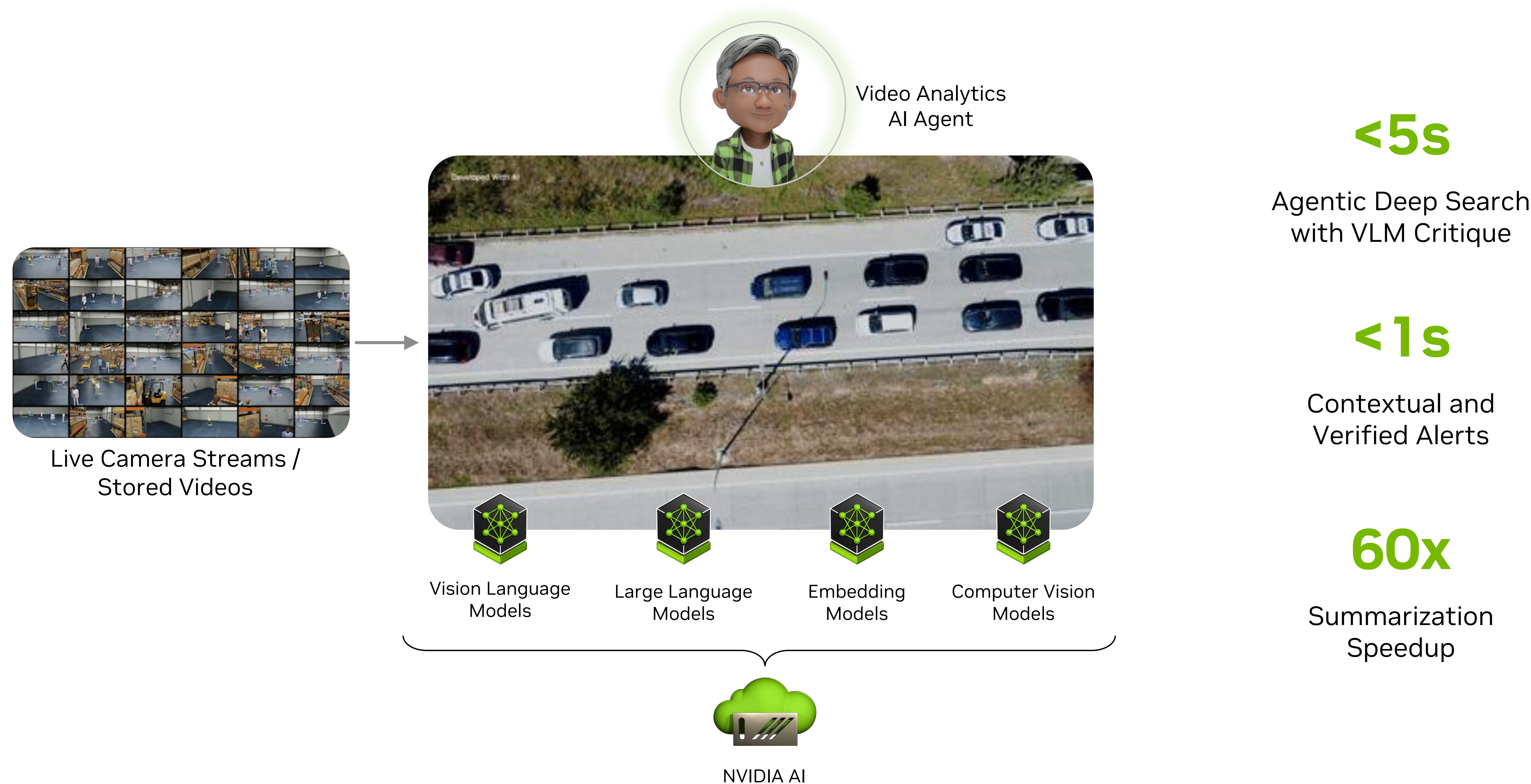
Agenda

- VSS Overview
- Models
- Vision Microservices
- Middleware
- Agent Workflows
- Agent Skills
- Summary



NVIDIA Metropolis Blueprint for Video Search and Summarization (VSS)

Build Video Analytics AI Agents to Analyze Billions of Live Video Streams and Petabytes of Data



VSS 3 Architecture

Framework for building Visual AI Agents

AI Models

- Detect, Track, Embed & Caption videos
- Finetune models in TAO

Vision Microservices

- Real time REST API controlled pipelines to extract metadata from video streams with AI models
- Build custom pipelines with DeepStream

Storage Services

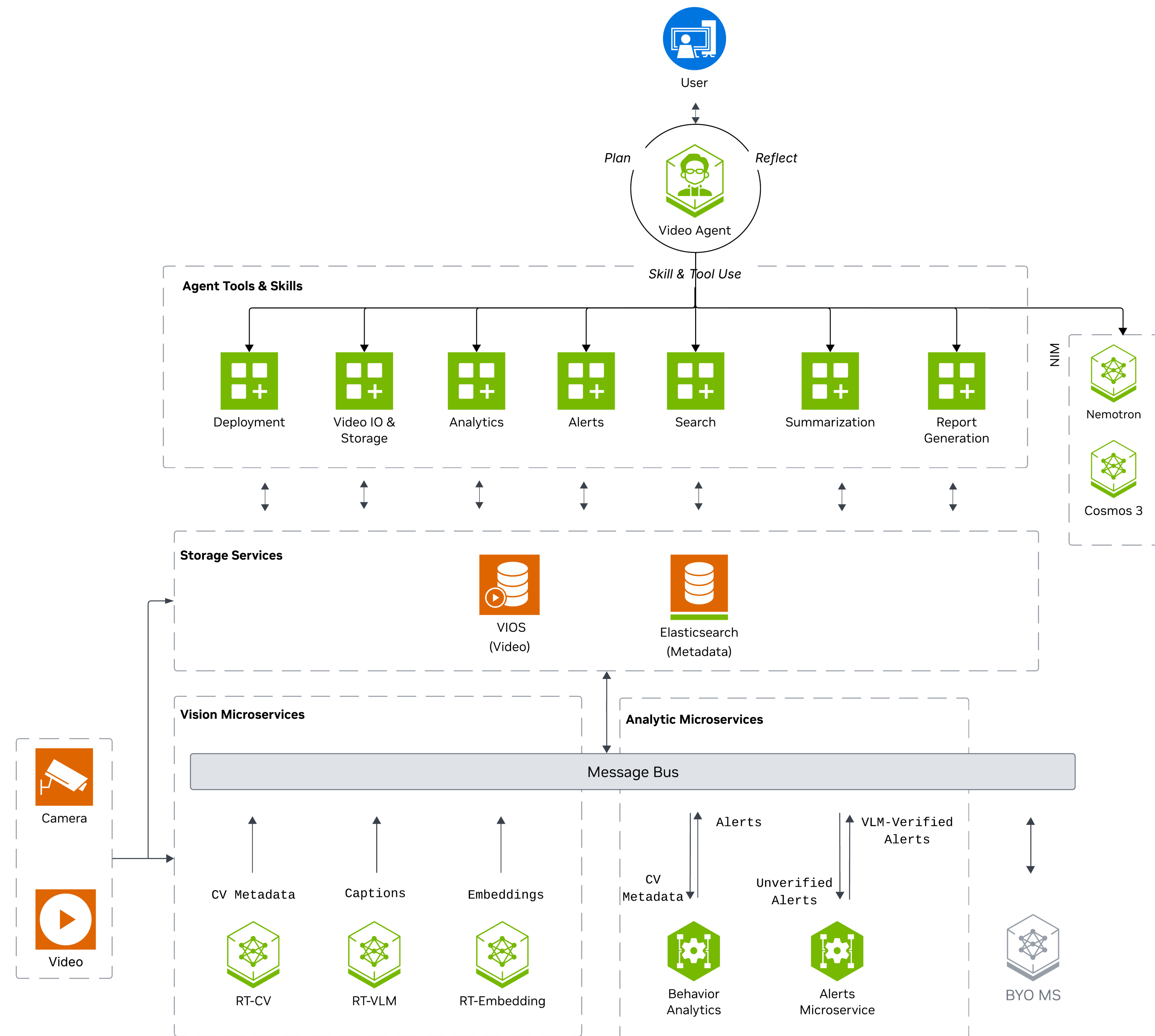
- VIOS – Video Database to manage, store, retrieve video streams and files
- Elastic Search – Database to store all extracted metadata, inference and analytic results

Analytic Microservices

- Operate on metadata in real time to extract further insights
- ROI/Tripwire Alerts, Multicamera Tracking, VLM Alerts

Agent Tools & Skills

- Pre-built tools for common usage and access patterns
- Skills for agentic use to deploy, build and interact with VSS



Models



Detection & Tracking

RT-DETR

INPUT

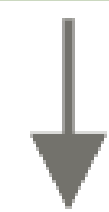
An RGB frame ($B \times 3 \times 640 \times 640$)



THE MODEL — RUNS IN RTVI-CV

RT-DETR

Detection Transformer for single-camera 2D detection
7 warehouse classes — person · forklift · pallet · AMRs · humanoids

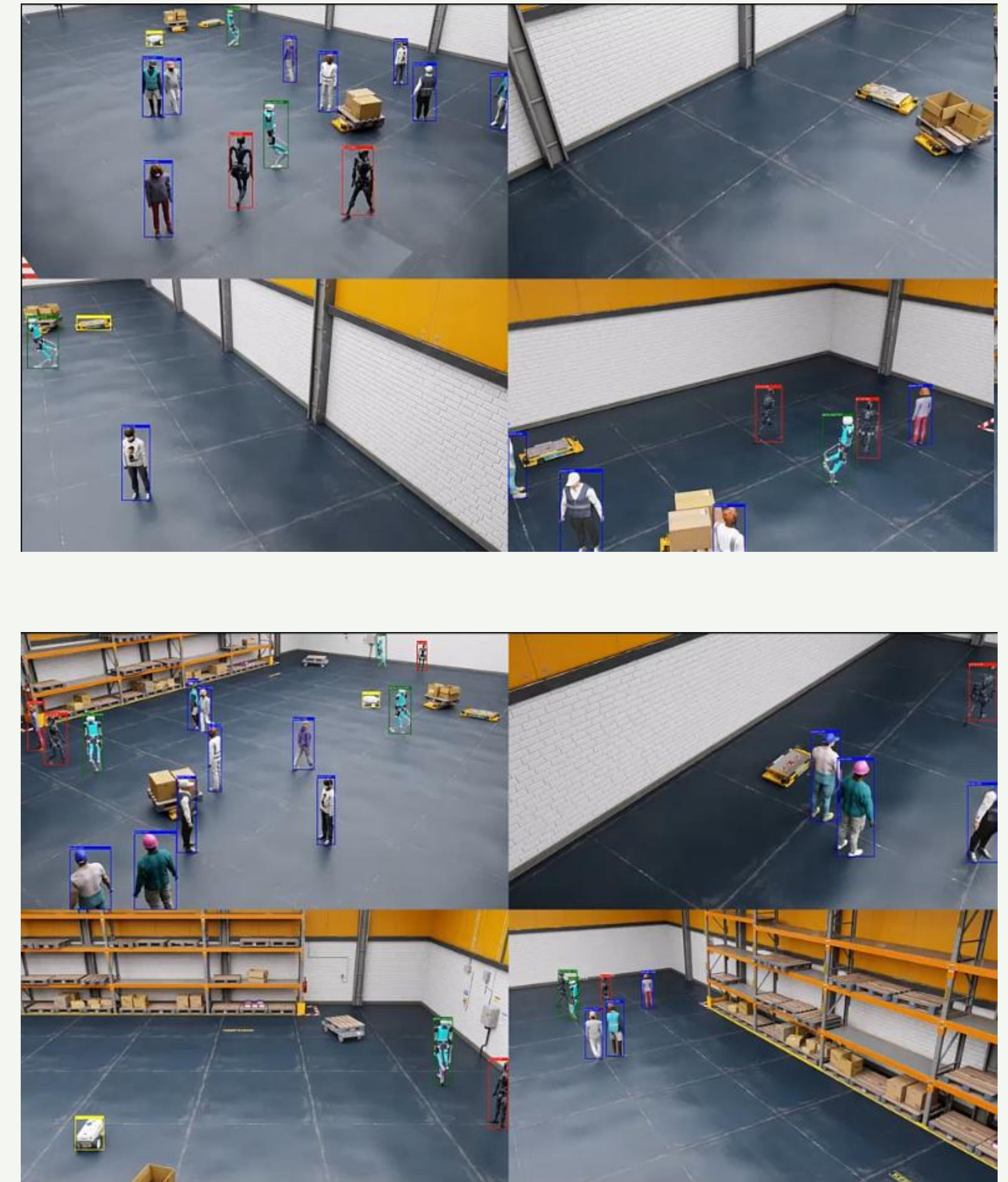


OUTPUT

2D bounding boxes + class + score

References

- <https://docs.nvidia.com/vss/3.2.0/models/rt-detr.html>
- [RT-DETR — Tao Toolkit](#)



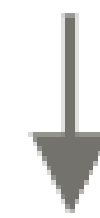
TAO RT-DETR sample output (warehouse 2D)

Detection & Tracking

Grounding Dino

INPUT

A frame + a text prompt — 'forklift' · 'spill' · 'open door'



THE MODEL — RUNS IN RTVI-CV

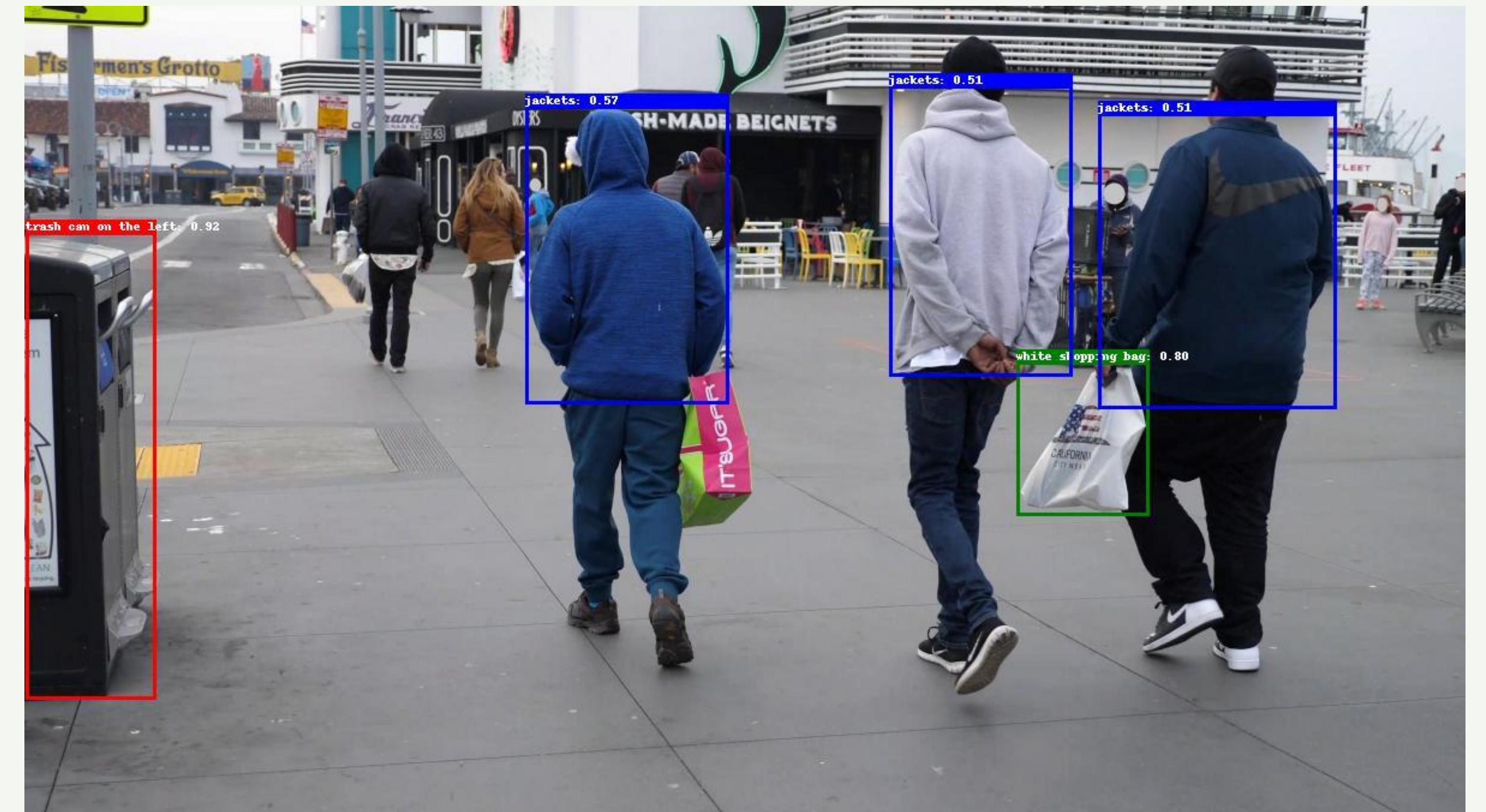
Grounding DINO

Open-vocabulary detection — no retraining for new classes
Swin-Tiny · served as ONNX / TensorRT (FP16)



OUTPUT

Boxes for the prompted classes + tracking IDs via DeepStream



Prompted with: 'jackets' · 'white shopping bag' · 'trash can', no retraining

References

- [Object Detection and Tracking — VSS](#)
- [Grounding DINO — Tao Toolkit](#)

Detection & Tracking

Sparse-4D: 3D multi-camera detection + tracking

INPUT

Synchronized multi-cam frames ($3 \times 540 \times 960$) + calibration



THE MODEL — RUNS IN RTVI-CV

Sparse4D

Fuses many camera views into one bird's-eye-view (BEV) space
Backbones: ResNet-50 (default) · ResNet-101 (more accuracy)



OUTPUT

3D boxes + tracking IDs consistent across cameras

References

- [Sparse4D — VSS](#)
- [3D Multi Camera Detection and Tracking \(Sparse4D\) — VSS](#)
- [Sparse4D — Tao Toolkit](#)

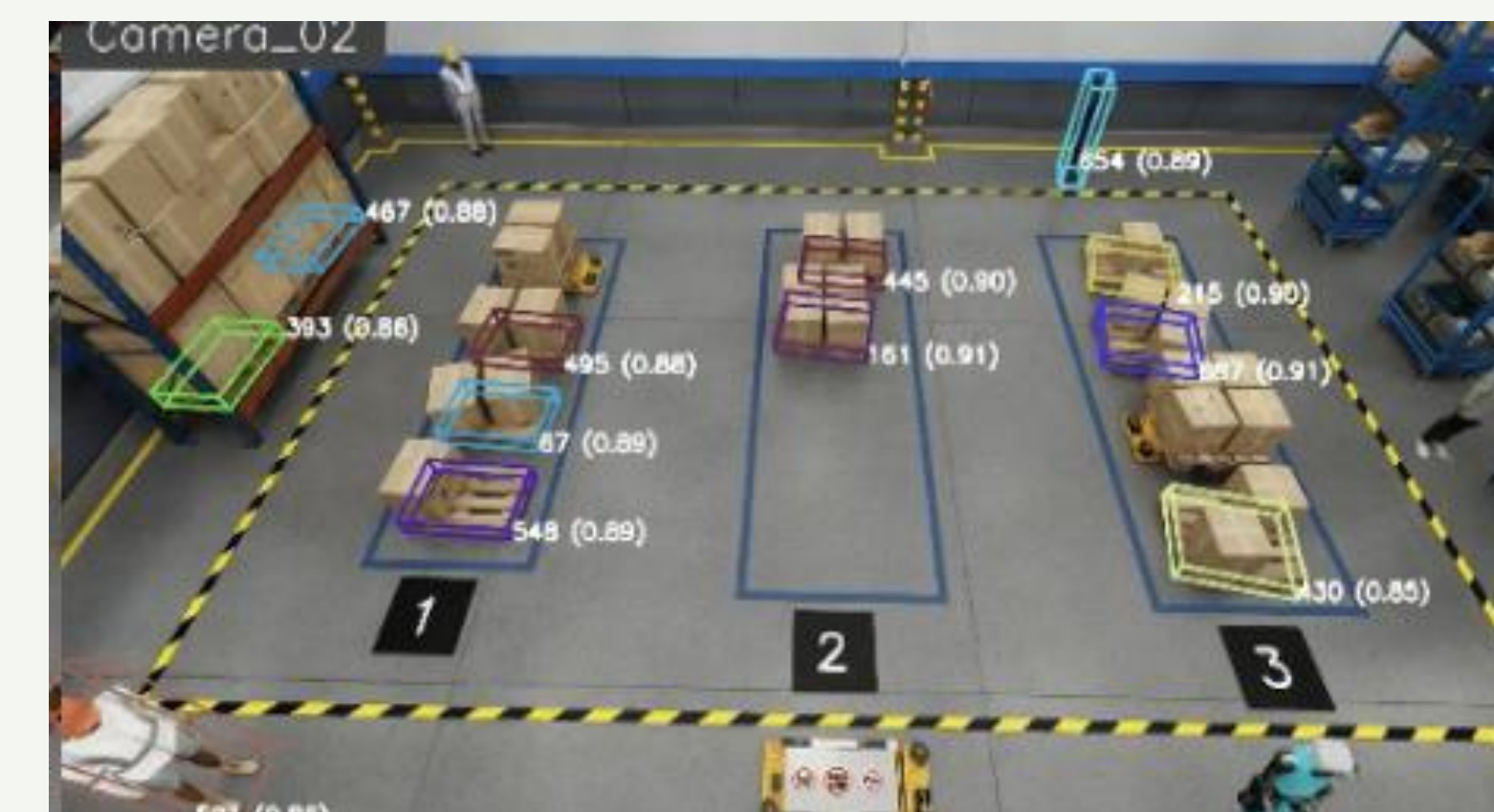
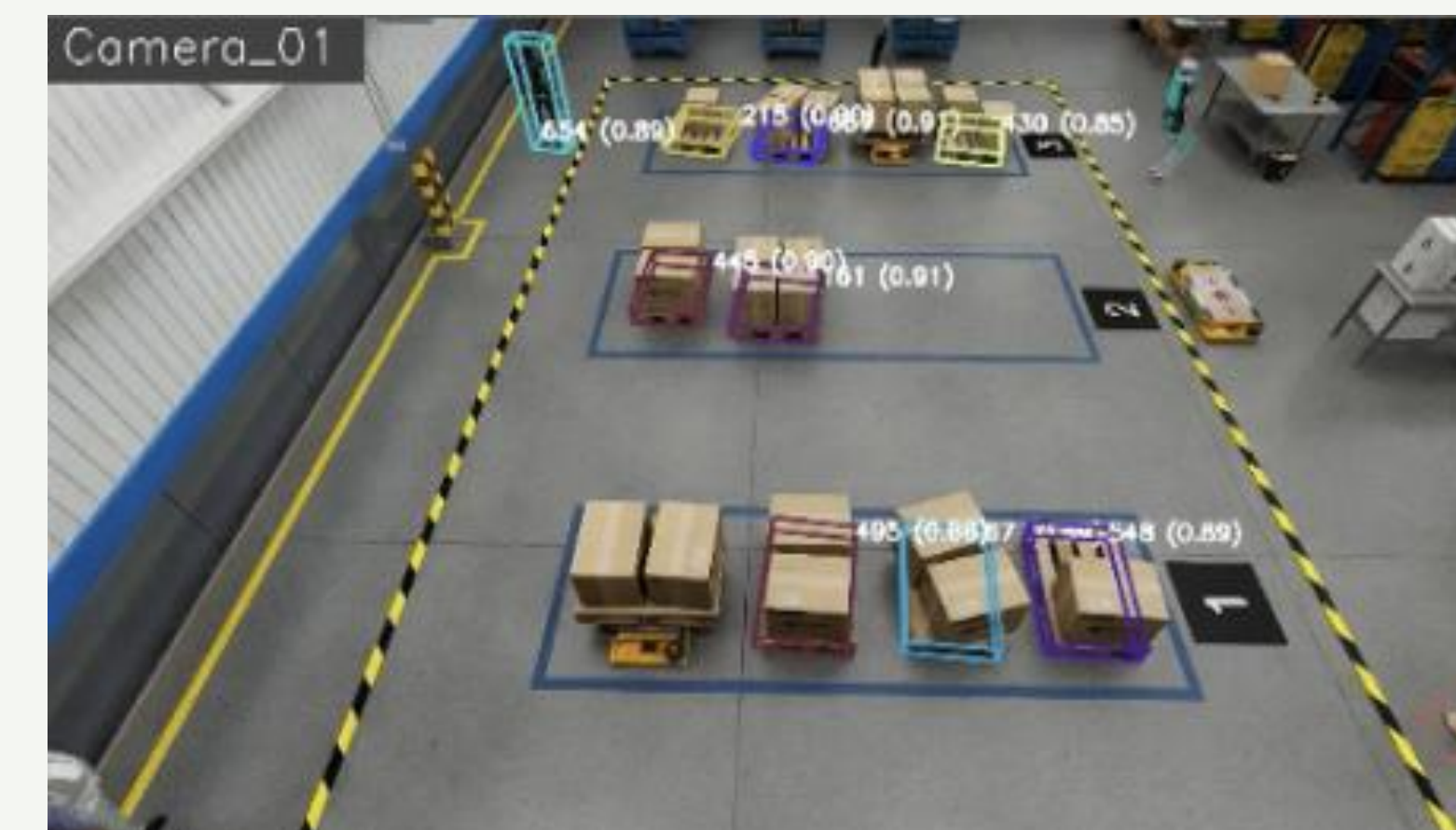


Image Embeddings

CLIP Based Models

PICK A MODEL — COMBINED IMAGE + TEXT ONNX

RADIO-CLIP

224 × 224 · 1024-dim
CLIPTokenizer (BPE)

SigLIP 2

SO400M-P16-256 · 1152-dim
256 × 256 · GemmaTokenizer

RTVI-CV LOADS IT ONCE — ONE FILE, TWO PLUGINS

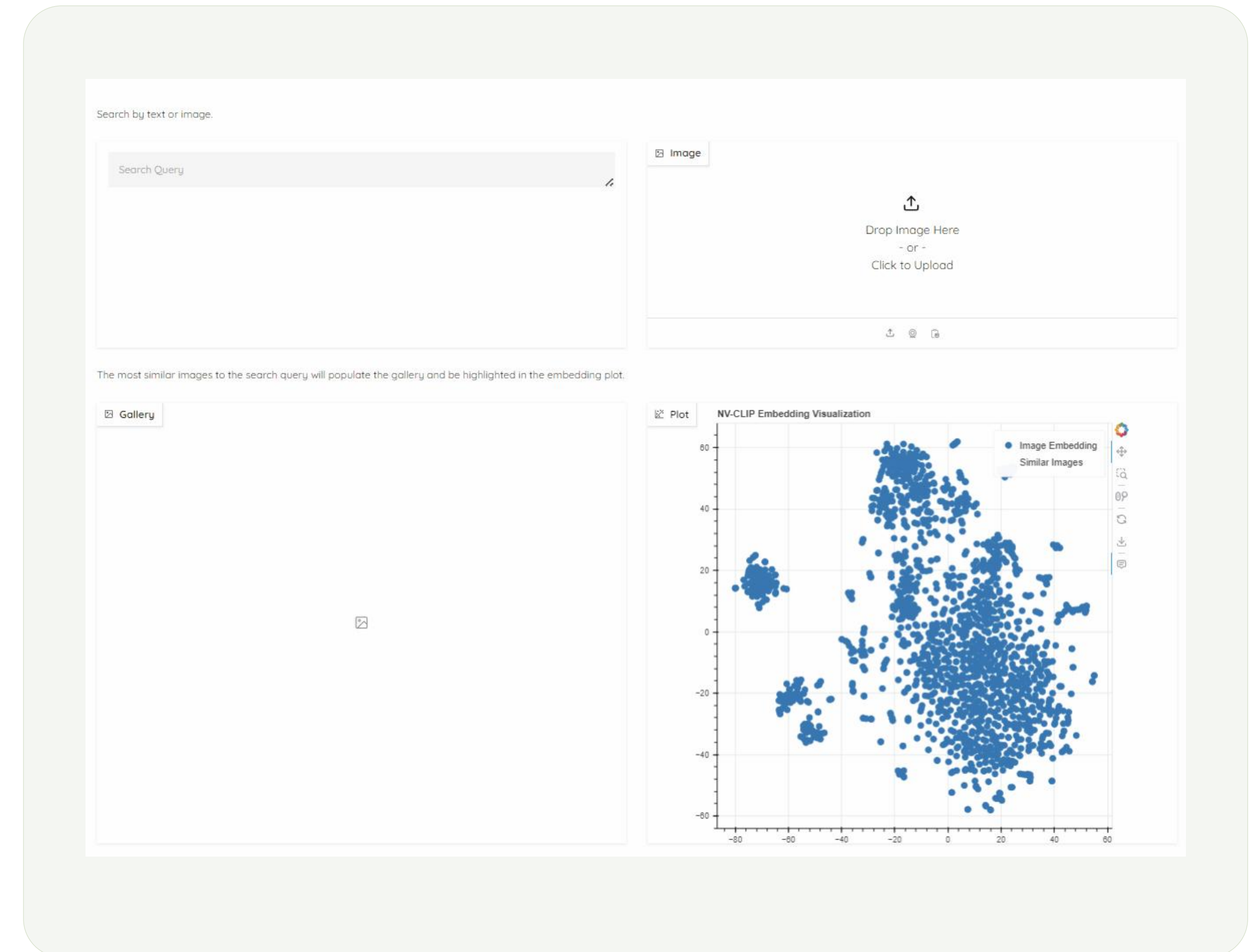
Combined image + text ONNX — one file, both encoders

VISION ENCODER · TensorRT

Object crops → embedding
ReID + object search

TEXT EMBEDDER · ONNX Runtime

Text → embedding
/generate_text_embeddings



References

- [SigLIP 2 object embeddings — VSS](#)
- [CLIP Introduction — Tao Toolkit](#)

Video Embeddings

Cosmos Embed

INPUT

An 8-frame video clip (1–2 FPS) + optional text (≤ 128 tokens)



THE MODEL — RUNS IN RTVI-EMBED

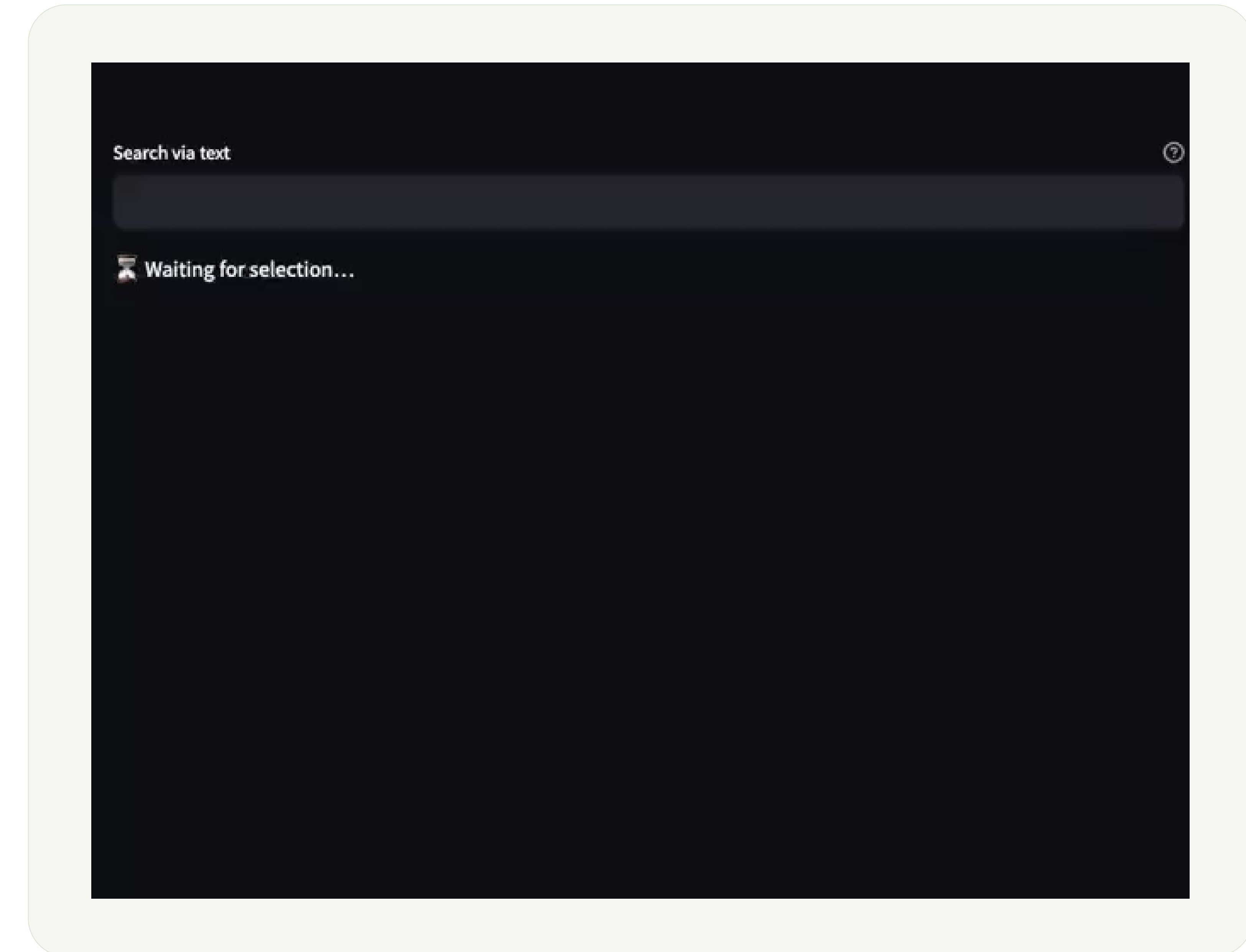
Cosmos-Embed1 (1B)

Joint video-text embedder — purpose-built for physical AI
Variants: 224p · 336p · 448p — trained on ~8M video-text pairs



OUTPUT

One normalized embedding — 256-dim (224p) · 768-dim (336p / 448p)

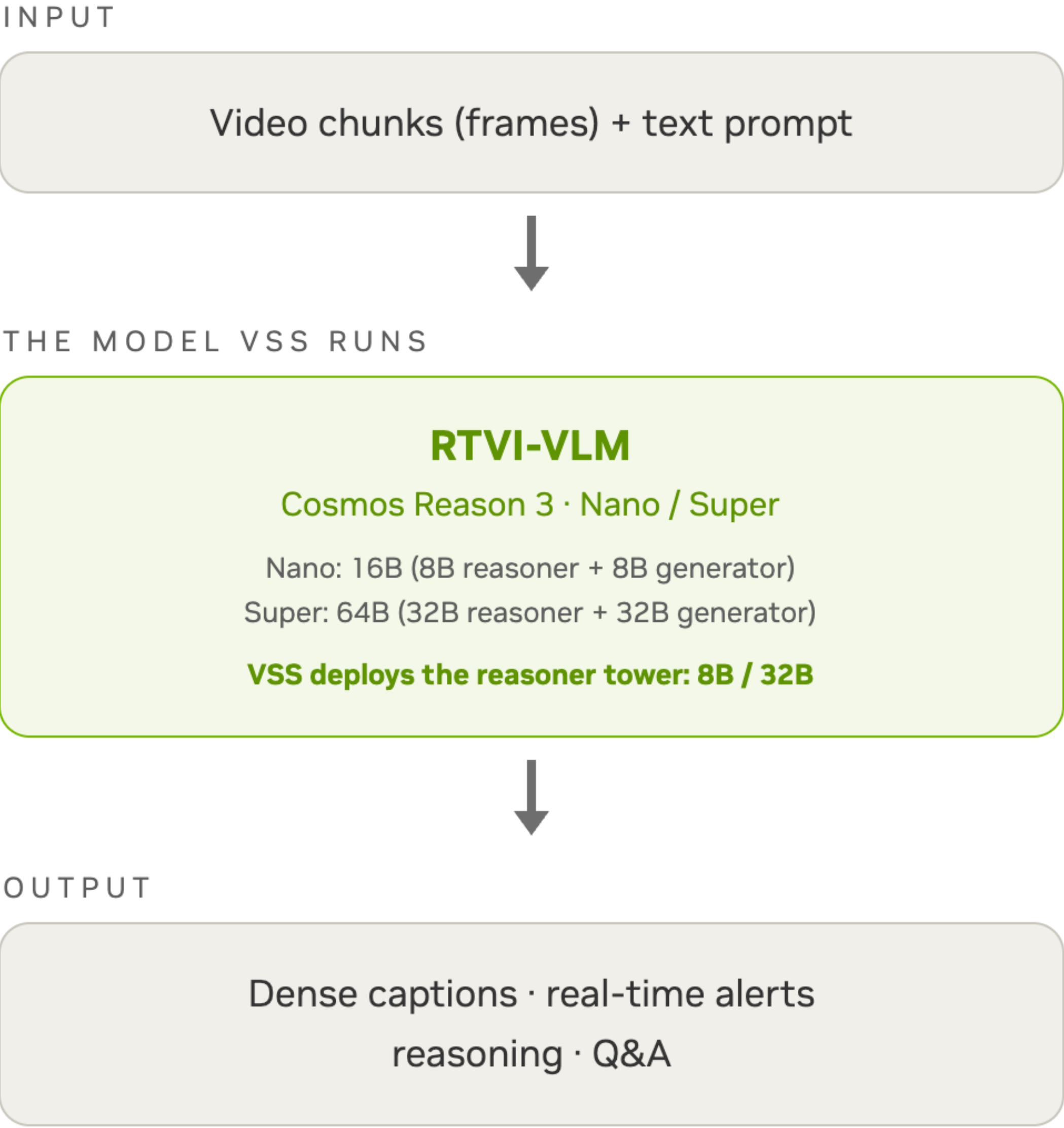


References

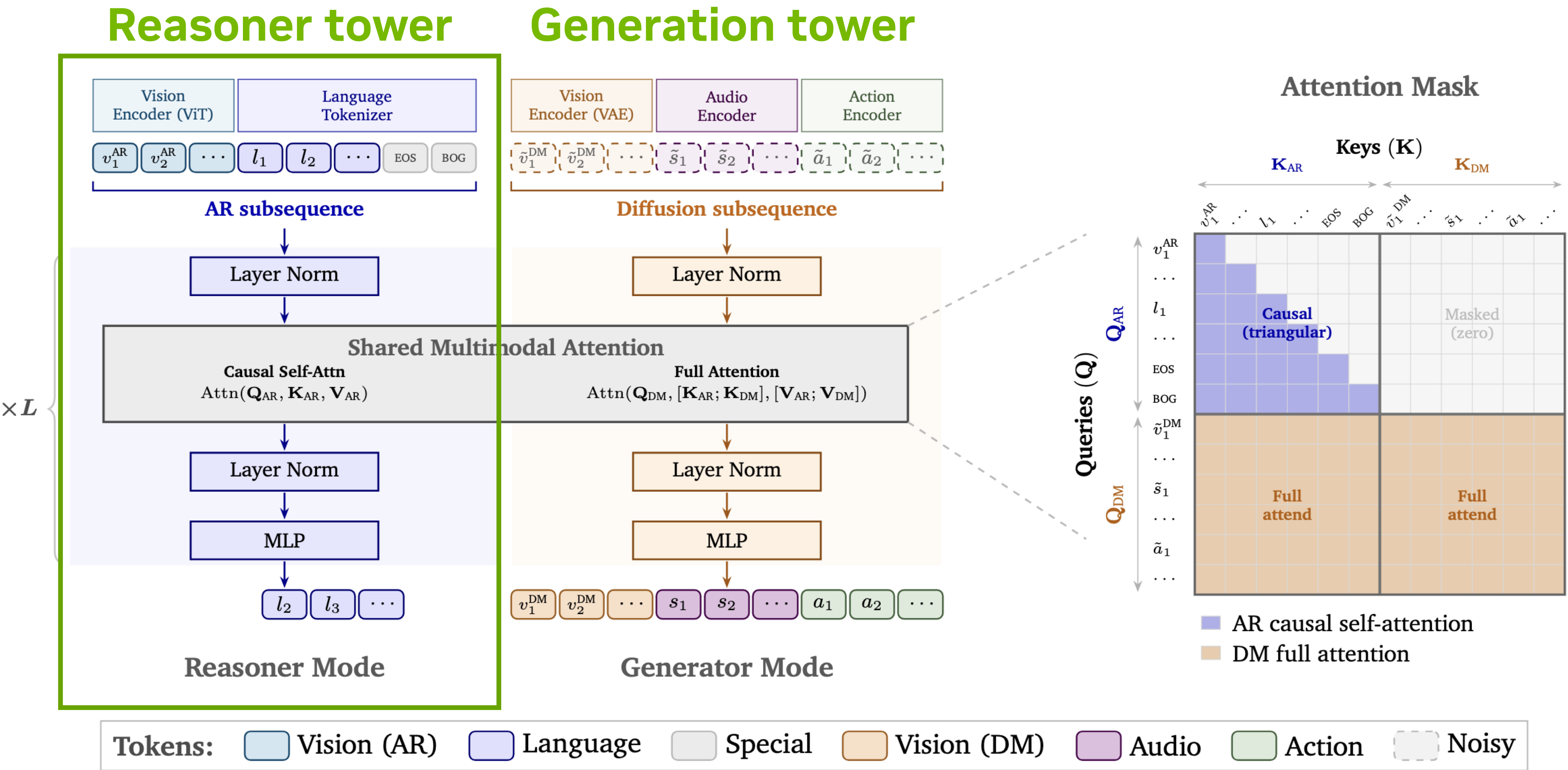
- <https://docs.nvidia.com/vss/3.2.0/models/cosmos-embed1.html>
- [Cosmos-Embed1 — Tao Toolkit](#)

Cosmos 3 in VSS: the reasoning VLM (Nano & Super)

VSS runs the Reasoner (autoregressive) tower as its VLM, the Generator (diffusion) and the World-Model / Simulator / WAM roles are context, not deployed in VSS.

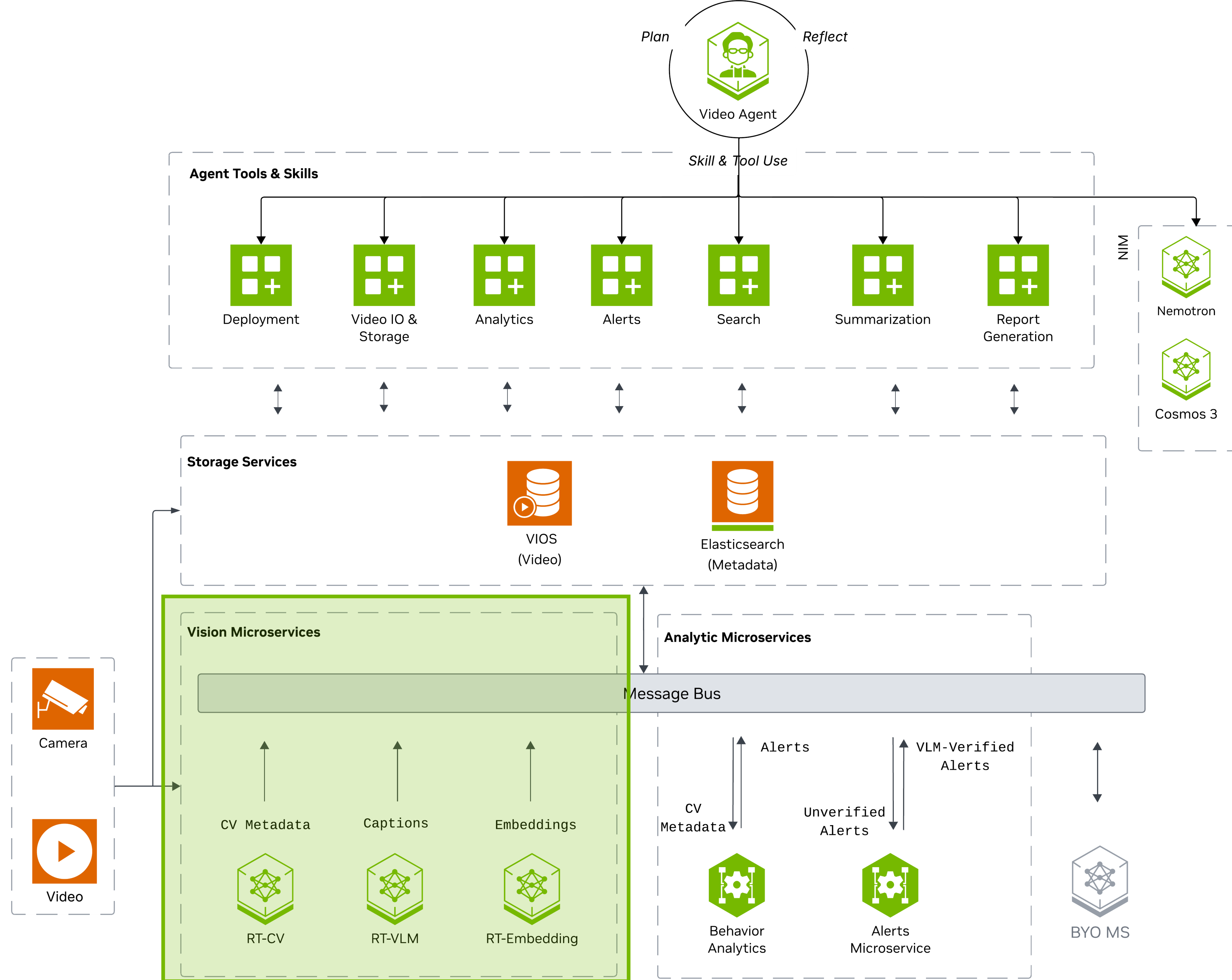


(0.3, 3.4): "A humanoid robot with a sleek white and black design stands beside a red popcorn dispenser filled with golden popcorn. The robot uses its right arm to pick up a green paper cup from the table in front of it, preparing to fill it."



Vision Microservices

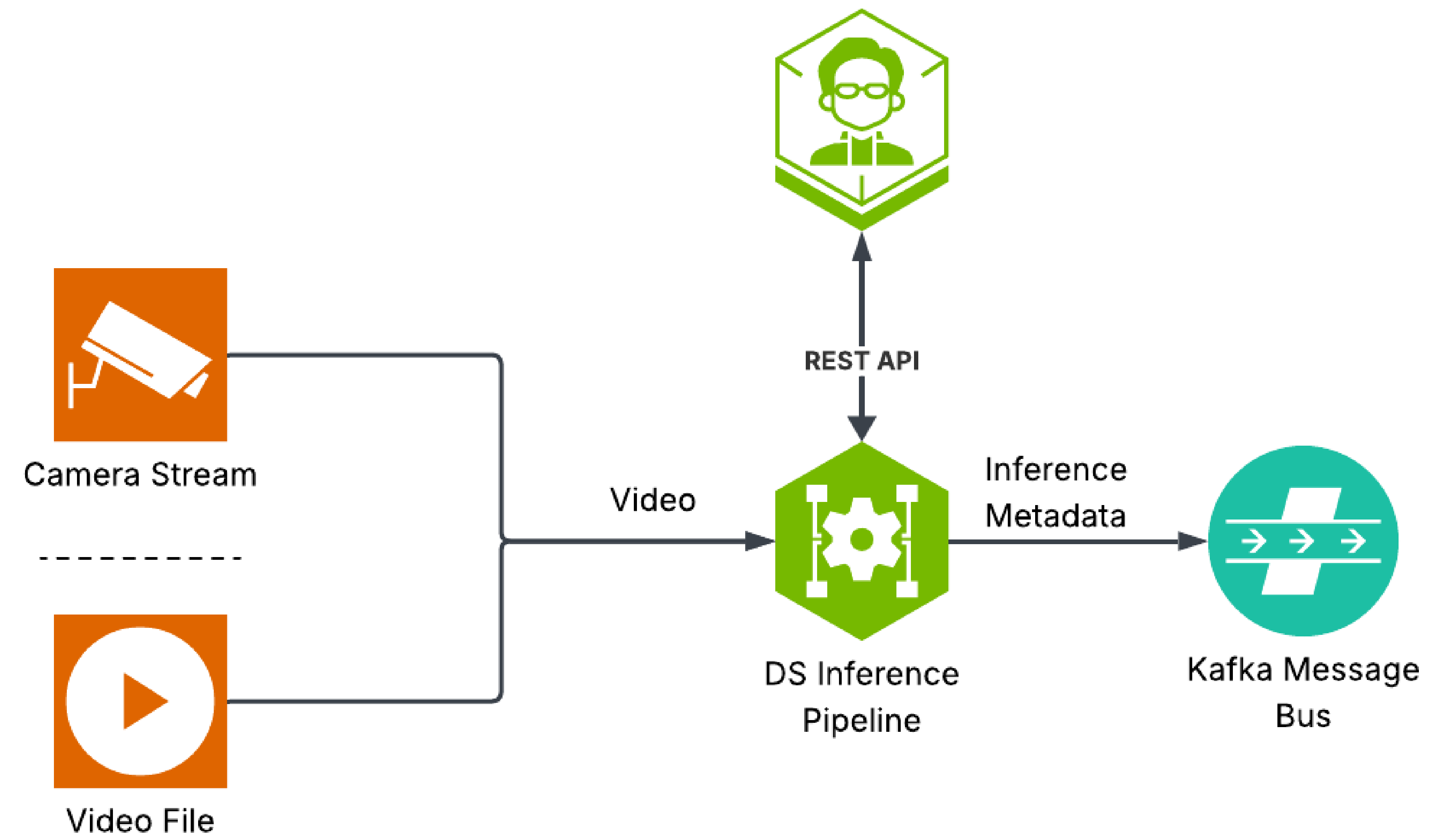




Vision Microservices Overview

What are they?

- DS Inference Pipeline for vision models controlled through REST API
- Input is RTSP stream or video file
- Input typically fetched from VIOS
- Output to Kafka message bus
- Used to extract real time insights into video data
 - Detection & Tracking
 - Embeddings
 - VLM Captioning
- Use finetuned models from TAO



References

- [Vision Microservices — VSS](#)

Object Detection and Tracking

Overview

2D Detections

- RT-DETR
- Grounding Dino

3D Detections

- Sparse4D

Detected Object Embeddings

- SigLIP 2
- RADIO-CLIP

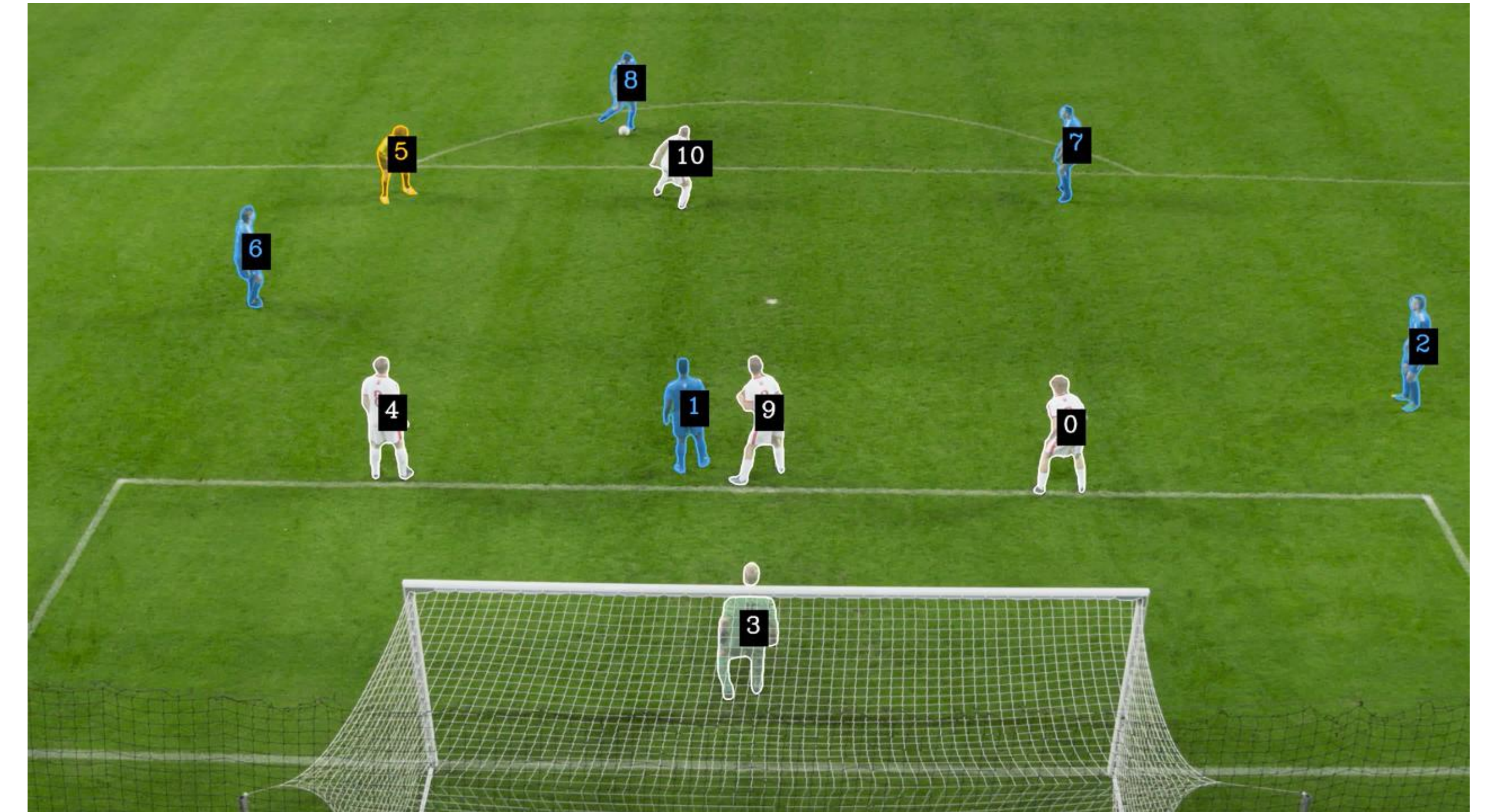
Input & Output

- Input: RTSP Streams, Video Files
- Output: Bounding Boxes, Tracking IDs, Embeddings

References

- [Object Detection and Tracking — VSS](#)

Detection & Tracking



Optional Embeddings

Detected Objects



Object Level Embeddings

[1.0121..., 0.9312..., 2.312..., ...]

[1.0121..., 0.9312..., 2.312..., ...]

Object Detection and Tracking

Usage – REST API Control

- Health & Monitoring Endpoints
- Stream Management Endpoints
 - Add/Delete RTSP Streams
 - Direct download links to video files also supported
 - Inference starts immediately after being added
- Outputs go to Kafka in protobuf

References

- [Object Detection and Tracking API Reference — VSS](#)

stream Stream management operations		^
POST	/api/v1/stream/add	Add a new video stream
POST	/api/v1/stream/remove	Remove an existing video stream
GET	/api/v1/stream/get-stream-info	Get information about active streams
health Health check and monitoring endpoints		^
GET	/api/v1/live	Liveness probe
GET	/api/v1/ready	Readiness probe
GET	/api/v1/startup	Startup probe
monitoring Metrics and telemetry endpoints		^
GET	/api/v1/metrics	Get application metrics
GET	/api/v1/metadata	Get application metadata

Object Detection and Tracking

Usage – Kafka Outputs

- Outputs go to Kafka in Protobuf format
 - Outputs per Frame metadata
 - Object Type
 - Bounding Boxes
 - Tracking ID
 - Optional Embedding
- Stored in mdx-raw and mdx-frames ES Index

References

- [Protobuf Schema — VSS](#)

```
{
  "timestamp": "2026-6-15T22:30:30.000Z",
  "sensorId": "Camera1",
  "objects": [
    {
      "type": ["Person"],
      "bbox.rightX": [984],
      "bbox.leftX": [958],
      "bbox.topY": [144],
      "bbox.bottomY": [165]
      "id": ["0"],
      "embedding": {"vector": [-0.3223, 0.4252, ....]}
    },
    {
      "type": ["Person"],
      "bbox.rightX": [324],
      "bbox.leftX": [234],
      "bbox.topY": [514],
      "bbox.bottomY": [812]
      "id": ["1"],
      "embedding": {"vector": [-0.5123, 0.3522, ....]}
    },
  ],
}
```

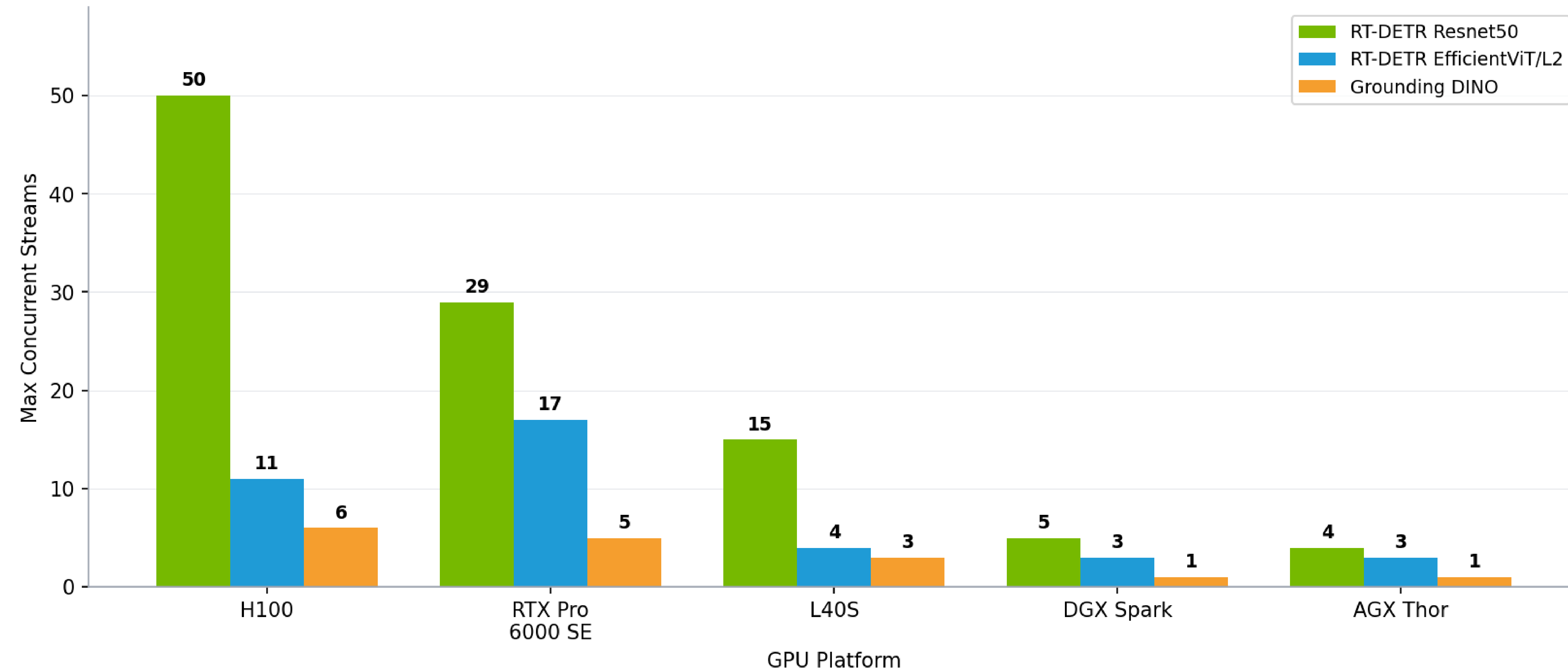
Object Detection and Tracking

Sizing

KPIs Measured

- Max Concurrent Streams at 30FPS
- Latency
- GPU Usage Metrics

RT-CV: Max Concurrent 1080p Streams per GPU



Throughput-maximizing mode · 30 FPS · 1080p · H.264 · FP16 · TensorRT · tracker enabled

References

- [RT-CV Performance — VSS](#)

Real-Time VLM

Overview

Supported Models

- Cosmos 1, 2 and 3 (Reasoner)
- Nemotron Omni
- Qwen 3 Omni
- Qwen 3 VL
- Any OpenAI Style VLM REST API Endpoint

Key Features

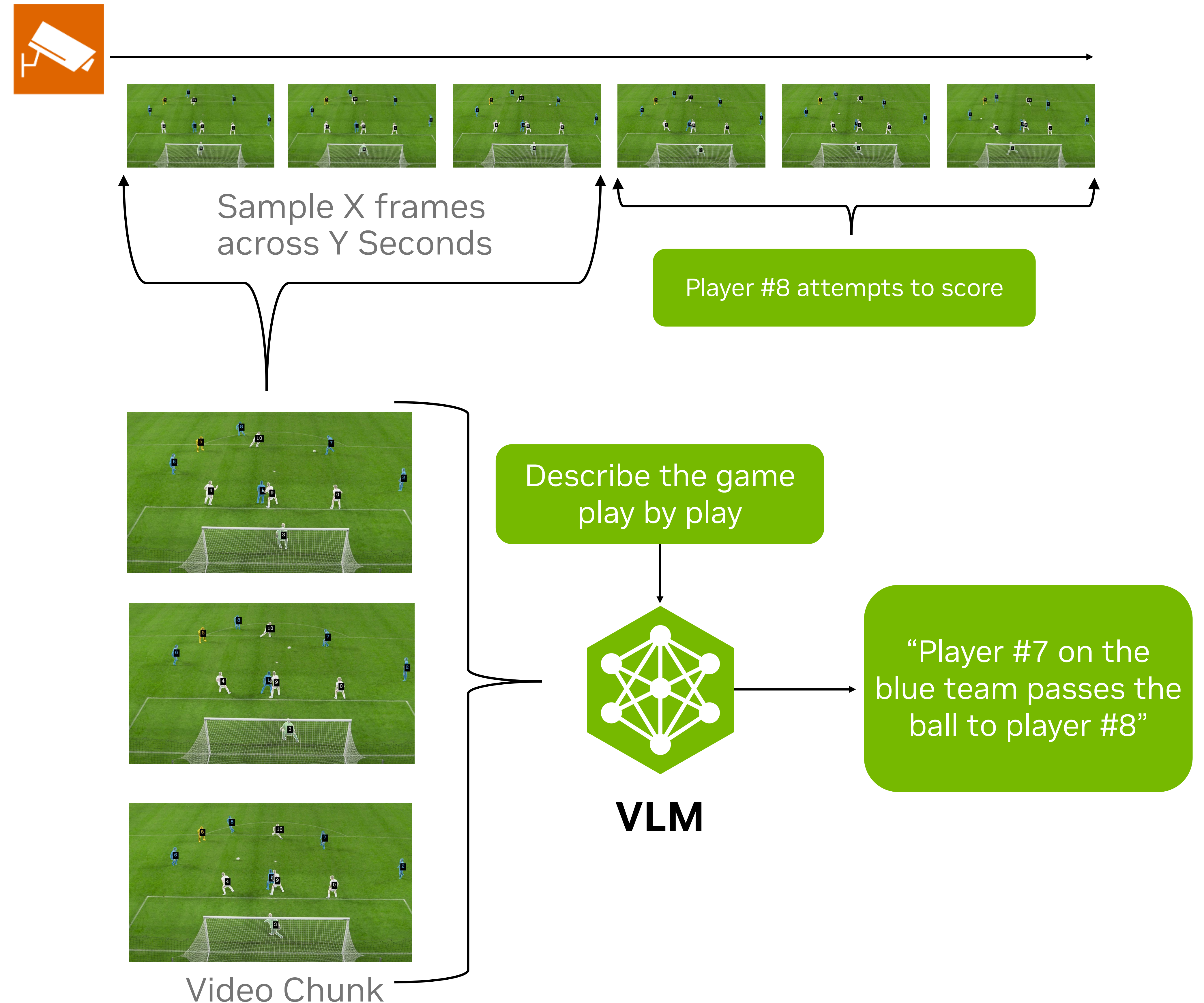
- GPU Accelerated decode, frame sampling, preprocessing & inference
- Audio transcription with Omni models

Input & Output

- Input: RTSP Streams, Video Files
- Output: VLM Responses

References

- [Real-Time VLM Microservice — VSS](#)



Real-Time VLM

Usage – Health Checks, Metrics & Stream Management

- Health Check & Metric endpoints
- Manage livestreams
- Video files supported

References

- [Real-Time VLM \(RTVI\) API Reference — VSS](#)

Health Check Operations to check system health.

GET	/v1/ready	Get RTVI VLM Microservice readiness status	🔒	✓
GET	/v1/live	Get RTVI VLM Microservice liveness status	🔒	✓
GET	/v1/startup	Get RTVI VLM Microservice startup status	🔒	✓
GET	/v1/assets/stats	Get asset storage statistics	🔒	✓
GET	/v1/health/live	Health Live (NIM compatible)	🔒	✓
GET	/v1/health/ready	Health Ready (NIM compatible)	🔒	✓

Metrics Operations to get metrics.

GET	/v1/metrics	Get RTVI metrics	🔒	✓
-----	-------------	------------------	---	---

Live Stream Operations related to live streams.

POST	/v1/streams/add	Add live stream(s)	🔒	✓
GET	/v1/streams/get-stream-info	List all live streams	🔒	✓
DELETE	/v1/streams/delete/{stream_id}	Remove a live stream	🔒	✓
DELETE	/v1/streams/delete-batch	Remove multiple live streams	🔒	✓
DELETE	/v1/generate_captions/{stream_id}	Stop a live stream from generating captions and alerts	🔒	✓

Real-Time VLM

Usage – Caption Generation & Inference

- Generate captions on added video files or livestreams
- Customize VLM parameters
- Customize frame sampling parameters
- Captions output as SSE (streaming request response) or Kafka
- Concurrent stream & caption support

References

- [Real-Time VLM \(RTVI\) API Reference — VSS](#)

Captions Operations to generate captions for a video.

POST

/v1/generate_captions

Generate VLM captions and audio transcripts for a video with alerts

system_prompt >

Expand all

string

≤ 10240 characters

matches ^(\.|\n)*\$

prompt* >

Expand all

string

≤ 10240 characters

matches ^(\.|\n)*\$

model* >

Expand all

string

≤ 256 characters

matches ^(\.|\n)*\$

max_tokens >

Expand all

integer

[1, 16384]

int32

min_tokens >

Expand all

integer

[1, 16384]

int32

temperature >

Expand all

number

[0, 1]

top_p >

Expand all

number

[0, 1]

top_k >

Expand all

number

[1, 1000]

chunk_duration >

Expand all

integer

[0, 3600]

int32

chunk_overlap_duration >

Expand all

integer

[-3600, 3600]

int32

num_frames_per_second_or_fixed_frames_chunk >

Expand all

number

[-1, 256]

float32

use_fps_for_chunking >

Expand all

(boolean | null)

vlm_input_width >

Expand all

integer

[0, 4096]

int32

vlm_input_height >

Expand all

integer

[0, 4096]

int32

enable_reasoning >

Expand all

boolean

Real-Time VLM

Sizing

Benchmarked for two scenarios

- **Captioning**
 - Long form descriptions
 - 100 token length responses
- **Alerting**
 - Single token responses to trigger notifications or check conditions

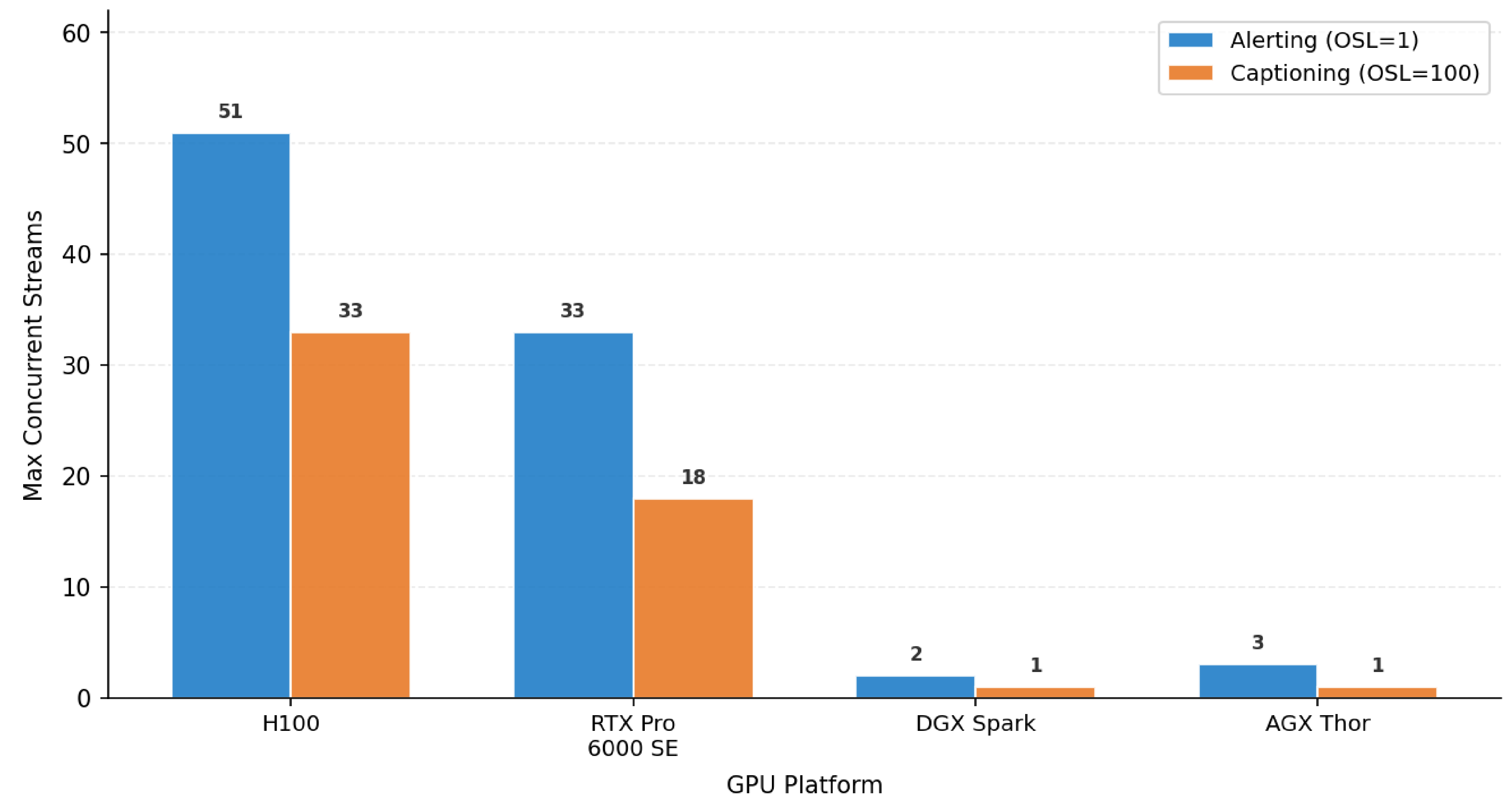
KPIs Measured

- Max Concurrent Streams
- Chunk Latency
- File processing latency

References

- [RT-VLM Performance — VSS](#)

RT-VLM: Max Concurrent Streams per GPU



Streaming mode · CR2-8B · FP8 · vLLM · 10-s chunks · 80 frames · 448×448

Video Embedding

Overview

Cosmos-Embed

- Joint Text + Video embedding model
- Variants
 - 448p
 - 336p
 - 224p

Key Features

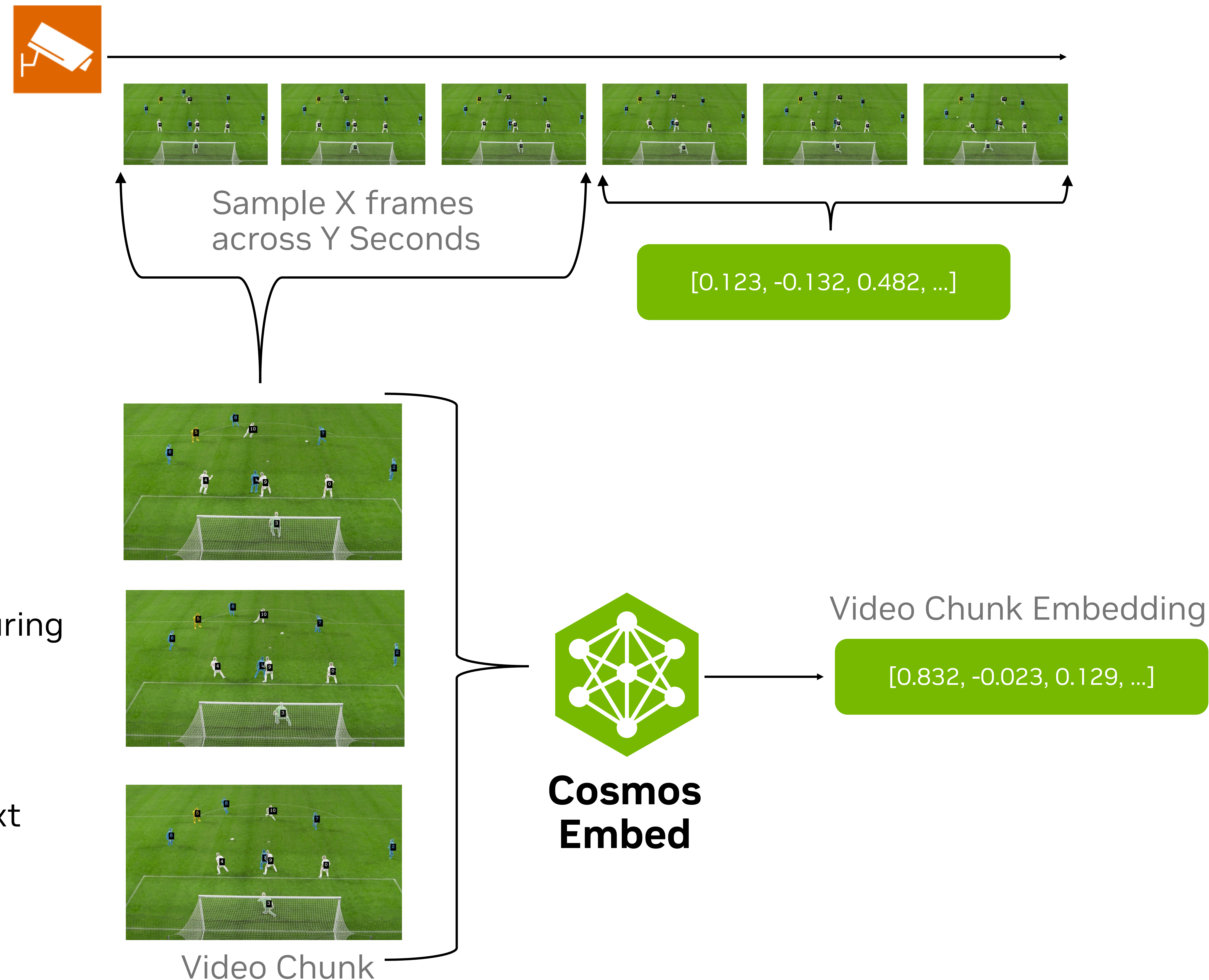
- GPU Accelerated decode, frame sampling, preprocessing & inference
- Vector based video search & retrieval. Best for capturing temporal information, actions & events

Input & Output

- Input: RTSP Streams, Video Files
- Output: Embedding vectors on video chunk and text prompts

References

- [Real-Time Embedding Microservice — VSS](#)



Video Embedding

Usage

- Add/Remove livestreams
- Upload/Delete video files
- Generate embeddings on added media
- Generate embeddings on text prompts for vector search
- Outputs received as SSE or on Kafka

References

- [Real-Time Embedding Microservice — VSS](#)

Embeddings Operations to generate embeddings for a video.

POST	/v1/generate_text_embeddings	Generate embeddings for a given text input.	🔒	⌵
POST	/v1/generate_video_embeddings	Generate embeddings for a video file, image, or live stream using specified chunk duration	🔒	⌵

Files Files are used to upload and manage media files.

POST	/v1/files	API for uploading a media file	🔒	⌵
GET	/v1/files	Returns list of files	🔒	⌵
DELETE	/v1/files/{file_id}	Delete a file	🔒	⌵

Live Stream Operations related to live streams.

POST	/v1/streams/add	Add live stream(s)	🔒	⌵
GET	/v1/streams/get-stream-info	List all live streams	🔒	⌵
DELETE	/v1/streams/delete/{stream_id}	Remove a live stream	🔒	⌵

Video Embedding

Sizing

- **KPIs Measured**
 - Max Concurrent streams
 - Average latency
 - GPU utilization metrics
 - Request Throughput

RT-Embedding: Max Concurrent Streams — Streaming Mode

Live RTSP · avg chunk latency must stay below 10 s threshold

GPU Platform	Max Concurrent Streams	Avg Latency (s)	GPU Util Avg (%)	NVDec Util Avg (%)
H100	170	5.19	63.74	94.59
RTX Pro 6000 SE	149	2.72	95.66	42.41

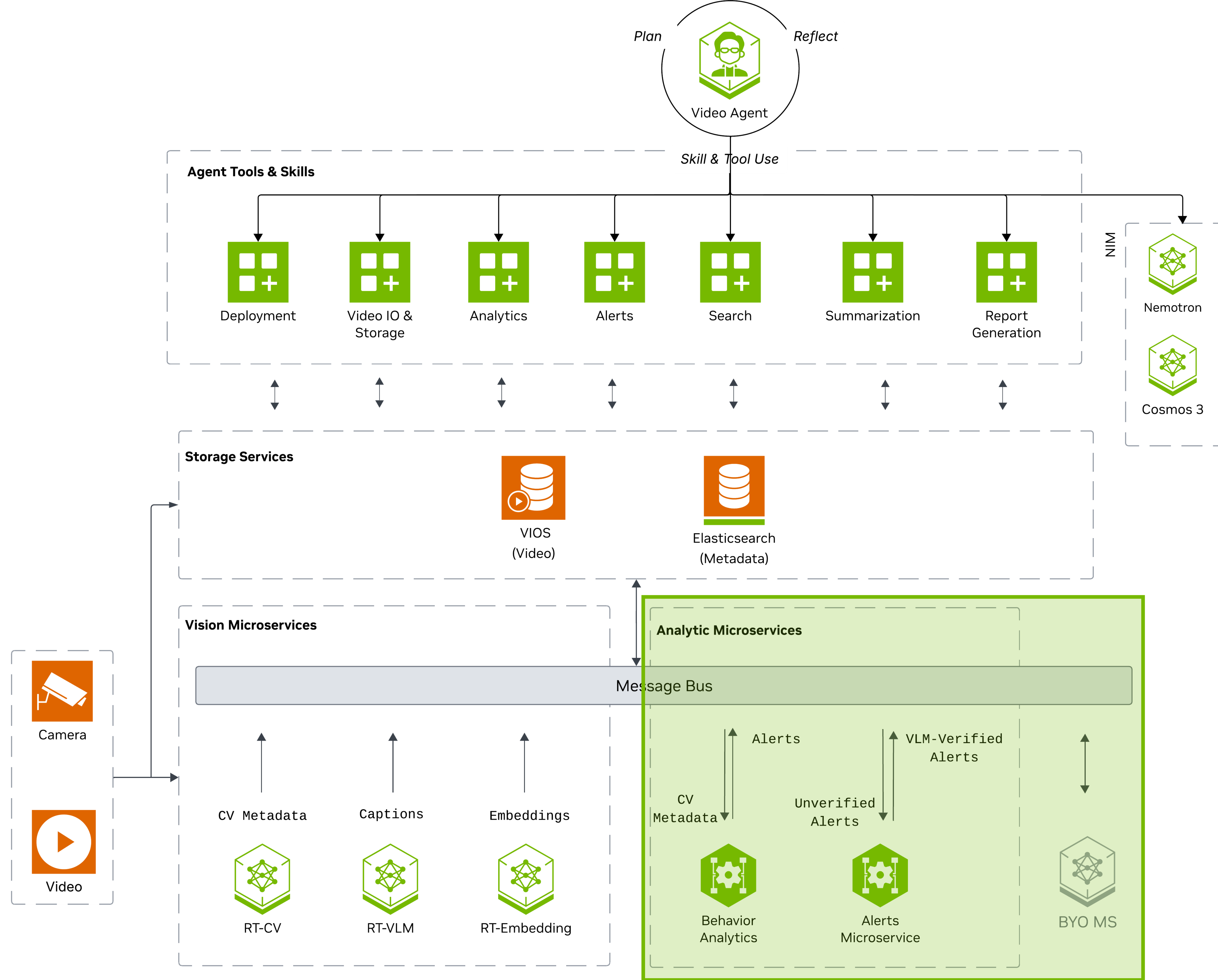
Cosmos-Embed1 · FP16 · TensorRT · 8 frames / 10-s chunk · 448×448 · 768-dim
Production recommendation: apply ~15% headroom (H100 → 144 streams, RTX Pro 6000 SE → 126 streams)

References

- [RT-VLM Performance — VSS](#)

Analytic Microservices

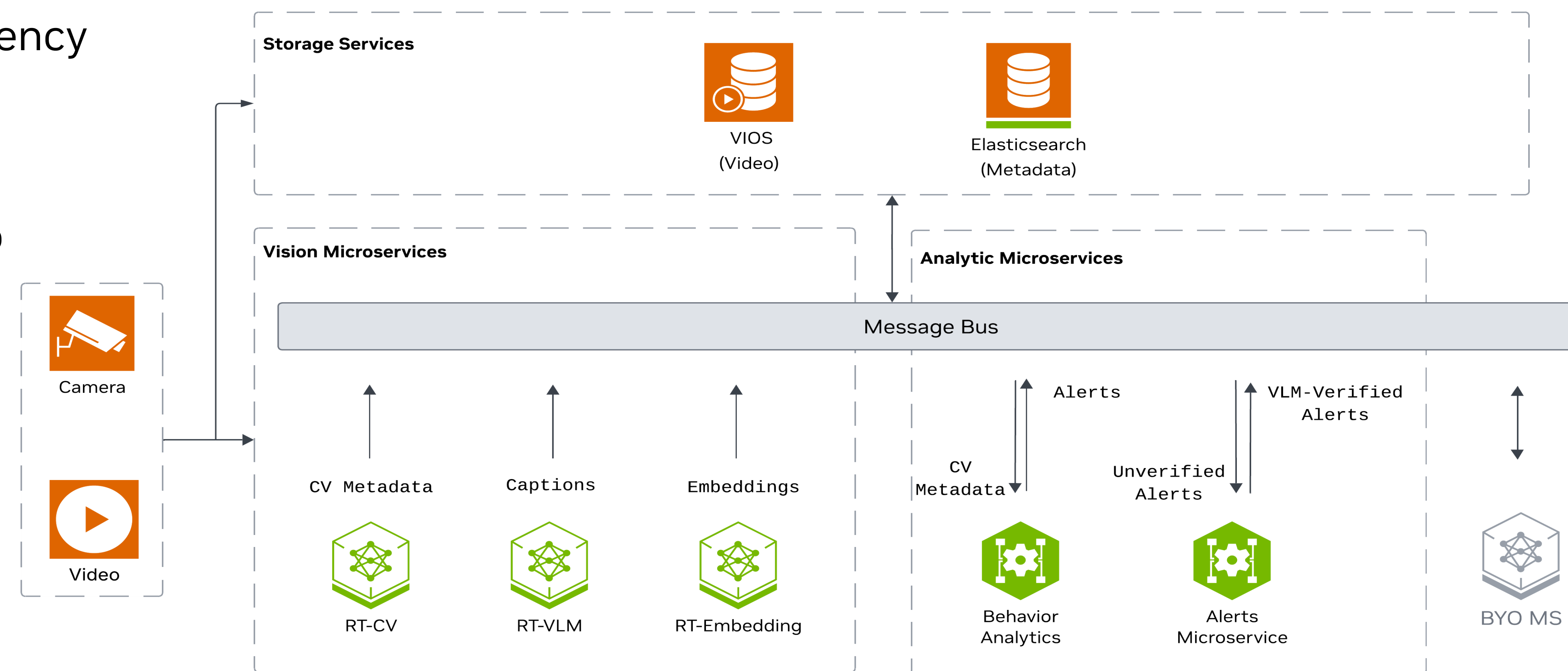




Analytic Microservices Overview

What are they?

- Microservices that postprocess inference metadata in real time to provide higher level insights
- Reads directly from message bus to receive low latency inputs
- Outputs new insights onto message bus
- Build your own microservices to post process video metadata



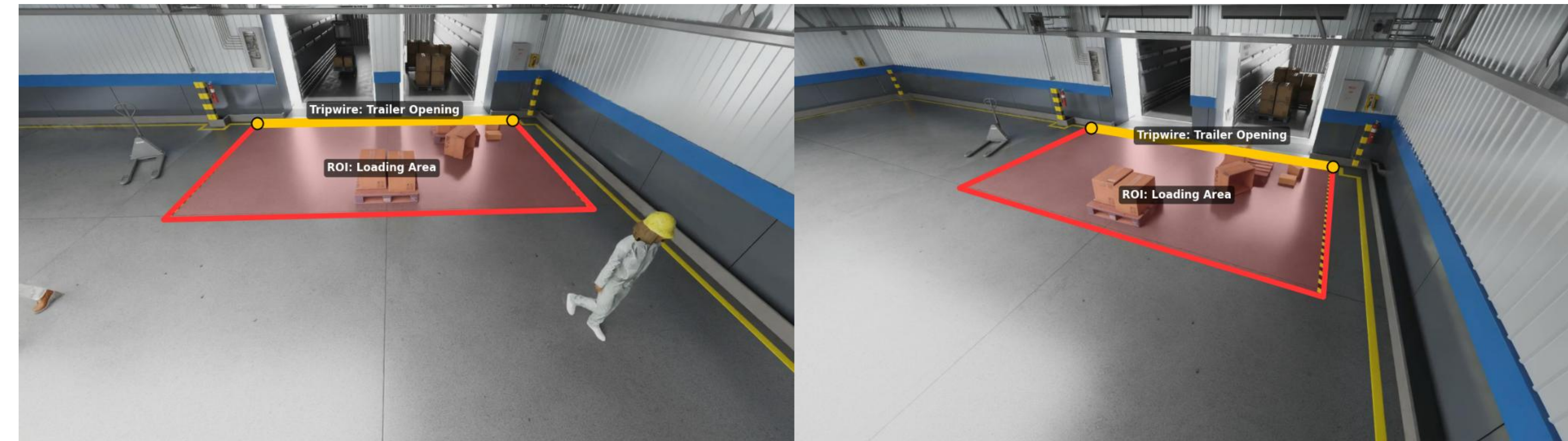
Behavior Analytics

Tripwires, Regions of Interest, Multi-Camera Tracking

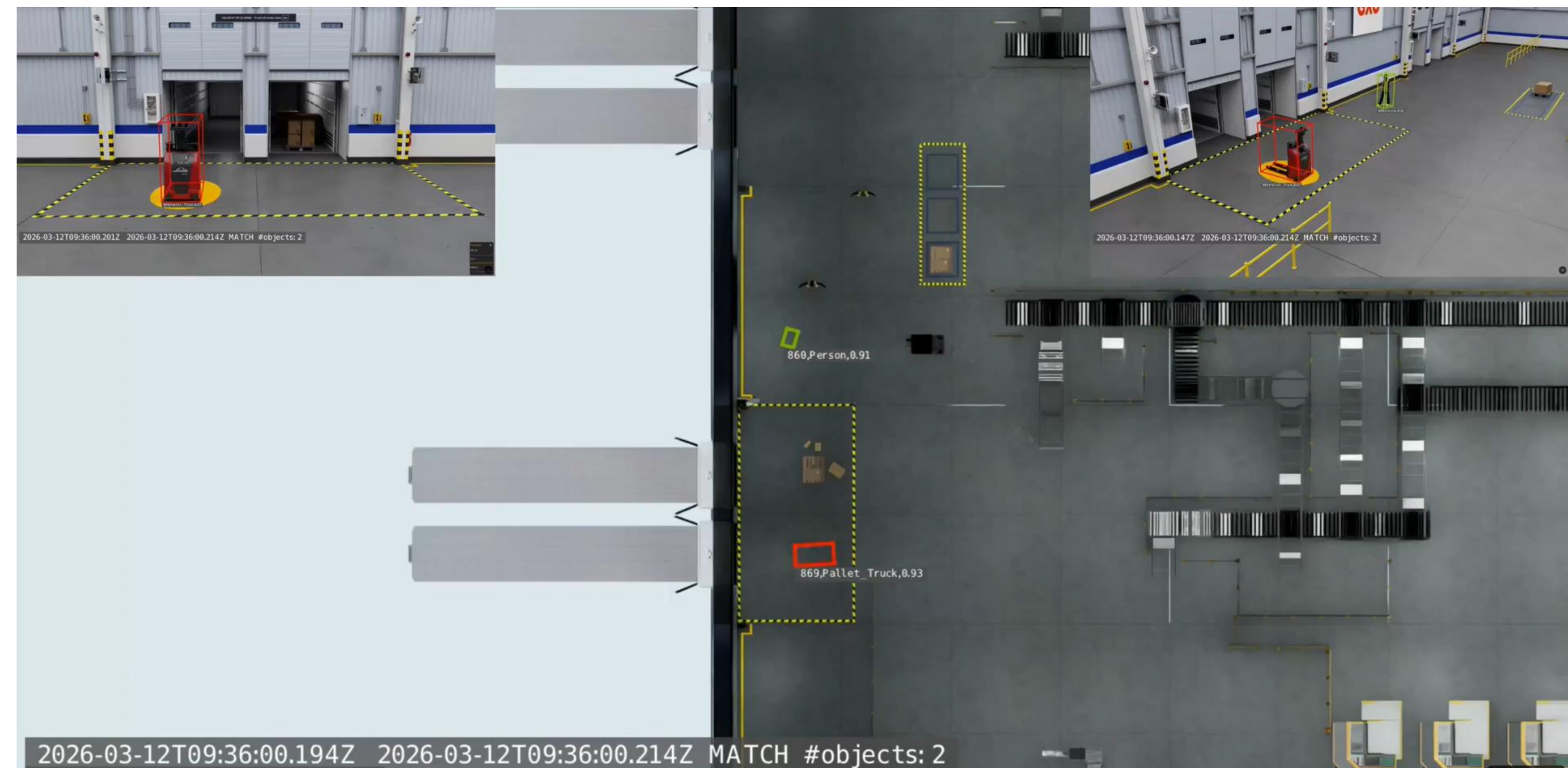
- Postprocesses detection & tracking data on single or multiple cameras for global scene understanding
- Requires camera calibration
- Converts pixel to real world coordinates
- Tripwires
- Regions of Interest
- Proximity alerts
- Multi-Camera tracking
 - 3D Profile
 - MV3DT

References

- [Behavior Analytics — VSS](#)
- [Warehouse Operations Blueprint Documentation — VSS](#)



Configure tripwires and regions of interest



Understand the scene in real world coordinates by fusing multiple camera angles

Alert Verification

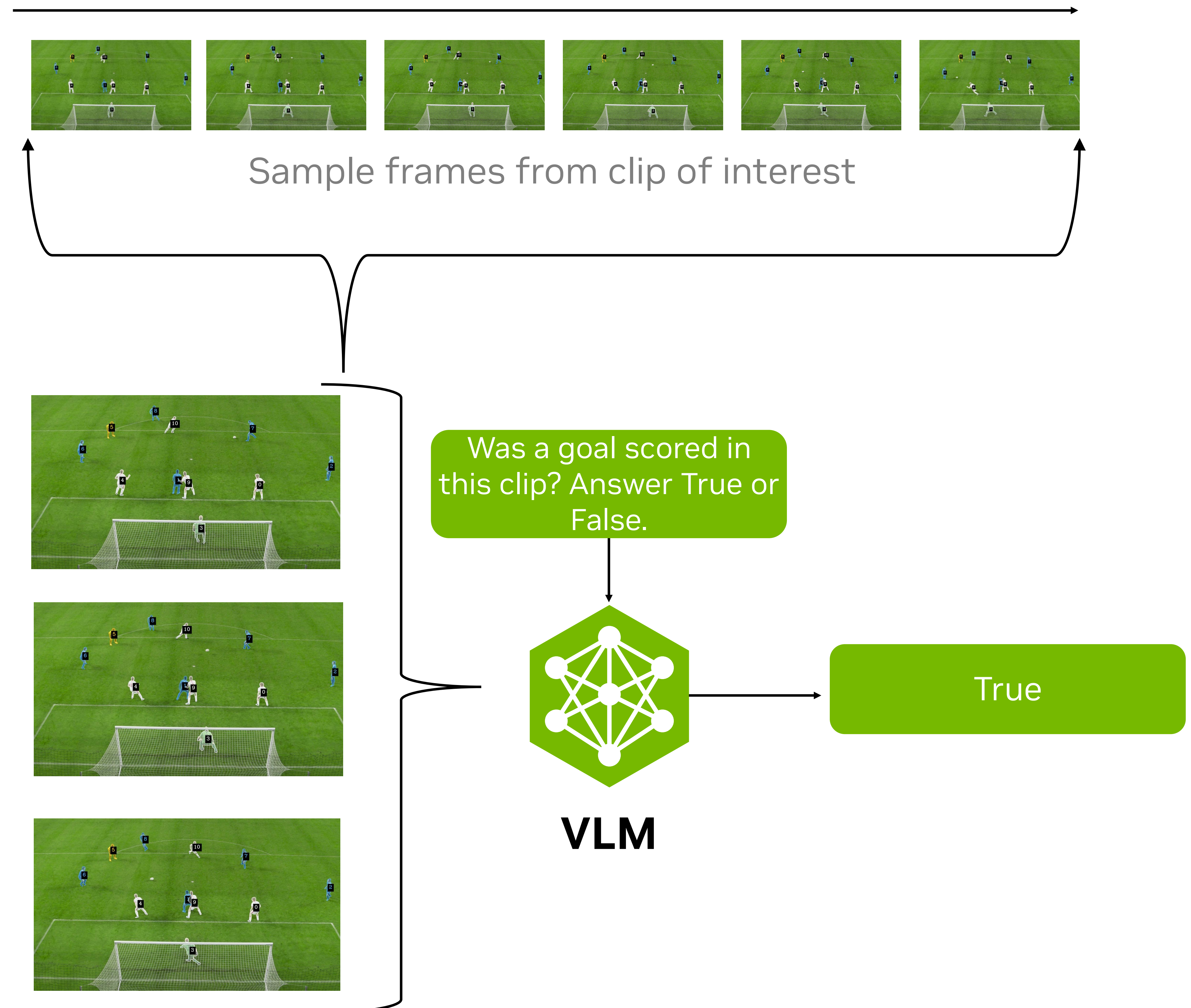
VLM Based Alerts



- **Alert verification, contextualization, and classification:** Process curated (short) clips from the VSS video pipeline (for example, RTVI CV and video analytics) through a VLM to confirm, enrich, or classify upstream alerts.
- **Real-time alerts:** Register video streams for continuous alert generation via periodic VLM inference through the RTVI VLM microservice.
- **Alert-on-demand:** Process short clips on demand through a VLM using preconfigured alert settings.

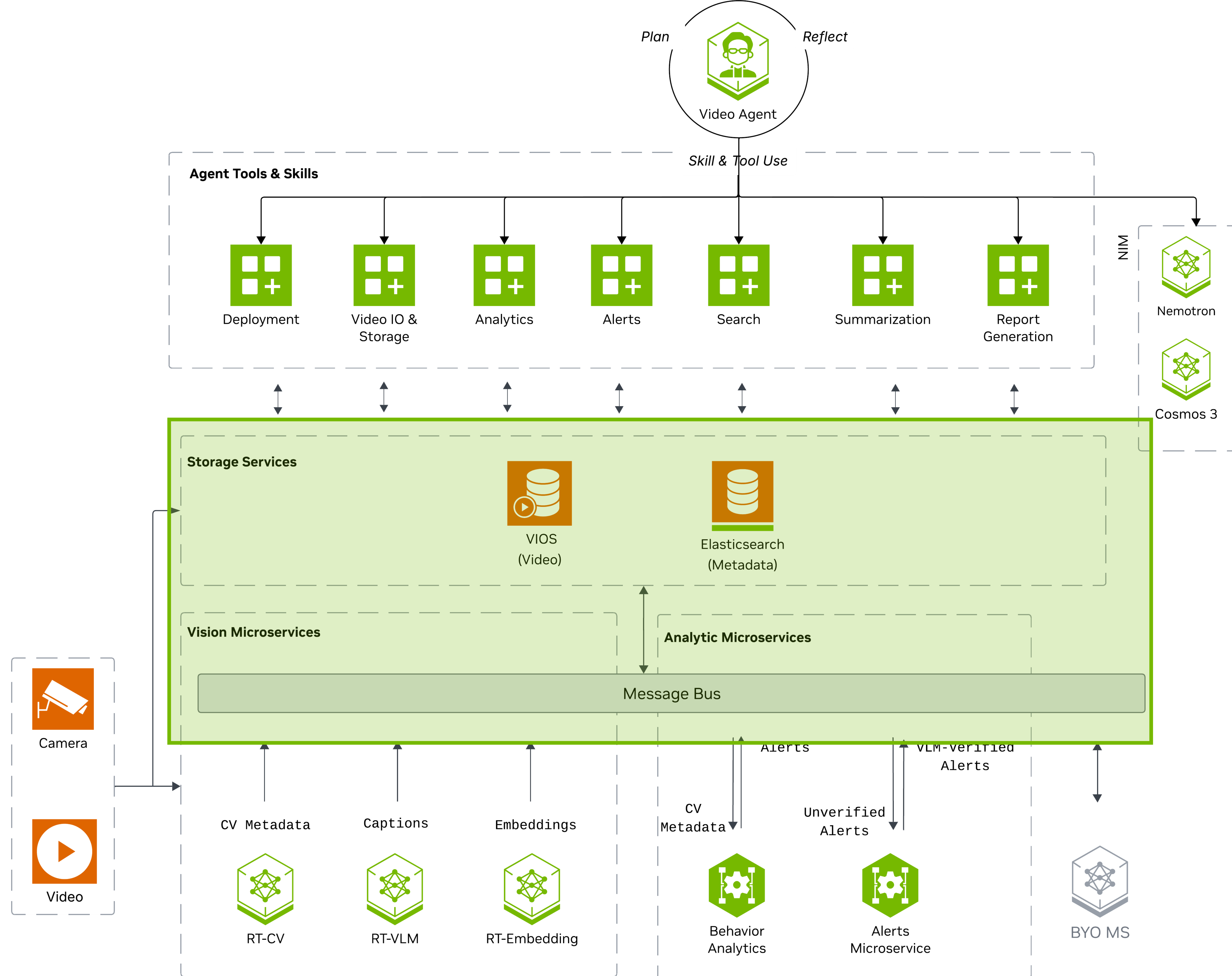
References

- [Real-Time VLM \(RTVI\) API Reference — VSS](#)



Databases & Middleware

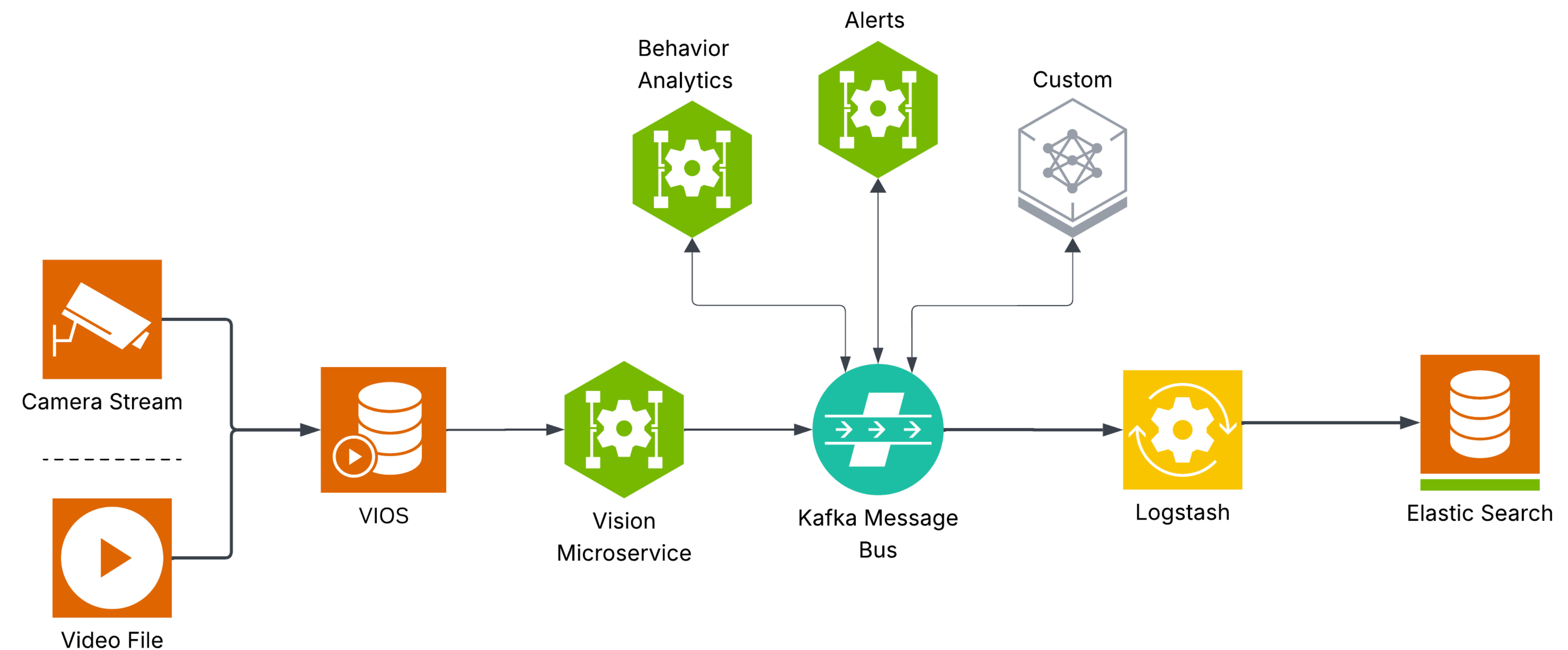




Middleware

What is it?

- Components that connect everything together to receive and store data
 - VIOS – Video I/O and Storage
 - Kafka – Message Bus
 - ELK Stack – ElasticSearch, Logstash, Kibana
- Vision Microservices publish metadata to a Kafka Topic
- Analytic Microservices subscribe and consume data from specific topics and post process the outputs
- Analytic Microservices publish new data to specific kafka topics
- Logstash reads and decodes all kafka topics and pushes the data into Elastic Search
- All data is timestamped and synchronized across the vision microservices, Kafka messages and databases
- Custom microservices can be implemented at this layer to receive low latency inference metadata by pulling directly from Kafka for post processing



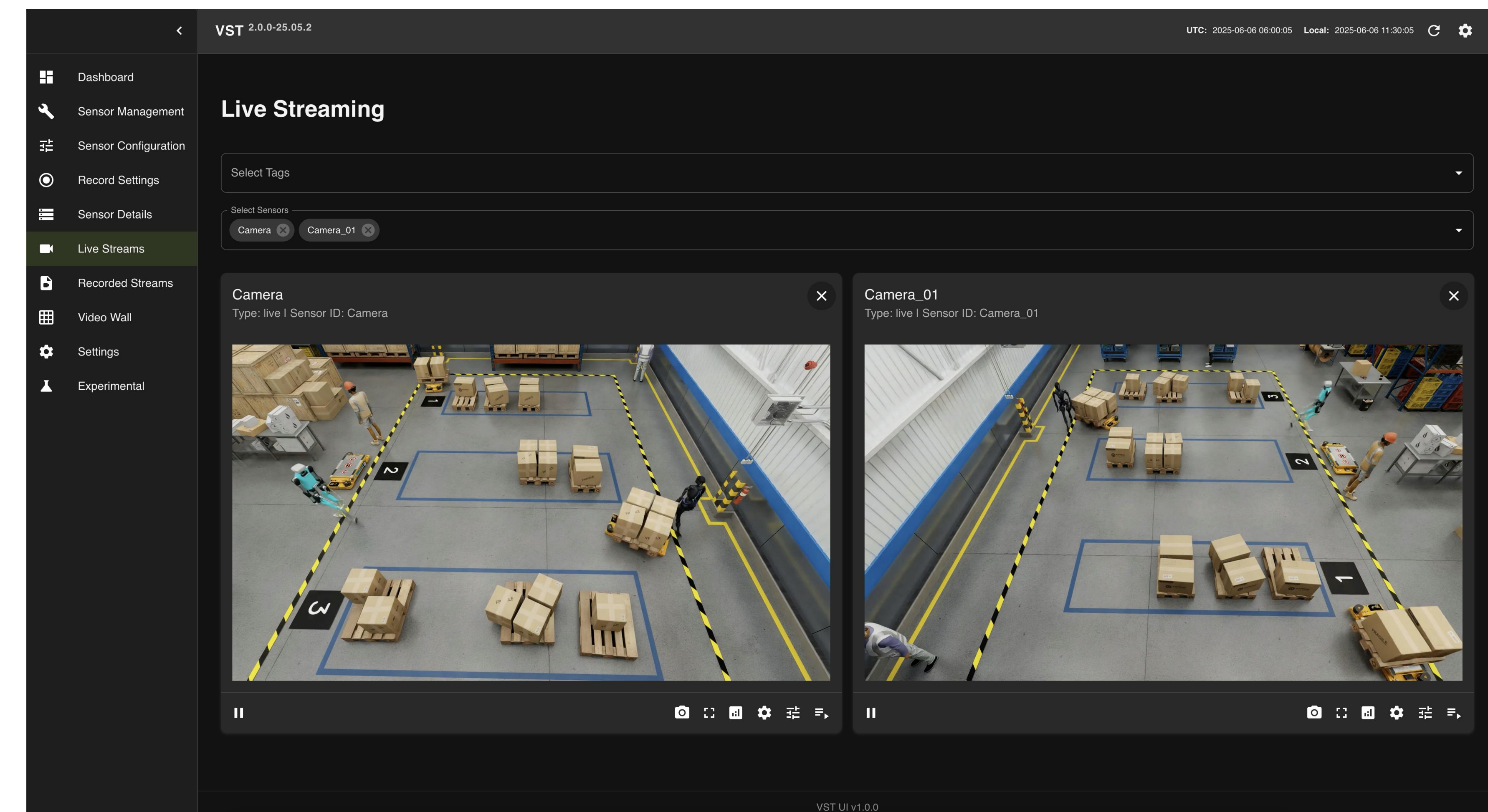
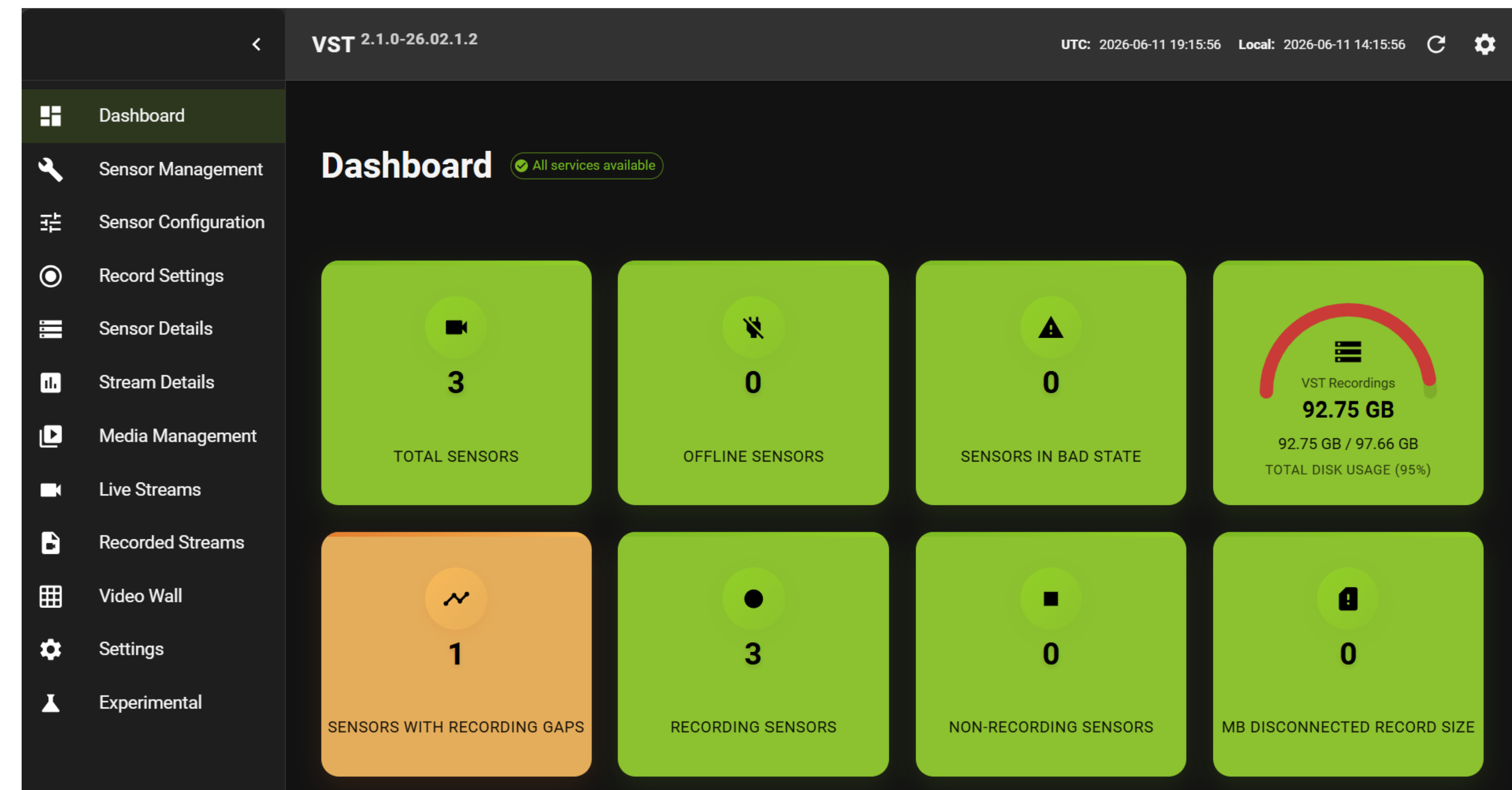
VIOS

Video I/O & Storage

- Formerly called VST (Video Storage Toolkit)
- Set of microservices for video & stream management
- Capabilities
 - Proxy RTSP Streams
 - Record Streams
 - Store video files
 - Retrieve clips based on timestamp
 - Generate streams from video files
 - WebRTC Output
- GPU accelerate decode/encode
- REST API Interface & Reference UI

References

- [Video IO & Storage \(VIOS\) Microservice — VSS](#)
- [NVStreamer — VSS](#)



VIOS

Usage

- Interact through REST API or UI
- Deploy through docker compose or prebuilt profiles
- Add/Remove Camera Streams
- Access live streams
- Start/Stop Recording
- Retrieve recorded clips between timestamps
- Access Proxy stream links
- Generate downloadable video links

References

- [VST Sensor Management API — VSS](#)
- [VST Record Stream Management API — VSS](#)
- [VST Storage Management API — VSS](#)

v1 VST Sensor Management API Schema v1			^
GET	/v1/sensor/list	Get list of sensors	▼
POST	/v1/sensor/scan	Scan sensors on the network	🔒 ▼
GET	/v1/sensor/{sensorId}/streams	Get streams of a sensor	▼
GET	/v1/sensor/streams	Get streams of all sensors	▼
POST	/v1/sensor/add	Add sensor manually	🔒 ▼
v1 VST Stream Recorder API Schema v1			^
GET	/v1/record/streams	Get list of available record streams	▼
POST	/v1/record/stream/add	Add stream to VST recorder MS	🔒 ▼
DELETE	/v1/record/{streamId}	Remove stream from VST recorder MS	🔒 ▼
POST	/v1/record/{streamId}/start	Start recording	🔒 ▼
POST	/v1/record/{streamId}/stop	Stop recording	🔒 ▼
v1 VST Storage Management API Schema v1			^
DELETE	/v1/storage/file/{streamId}	Delete media file associated with the provided ID	🔒 ▼
GET	/v1/storage/file/{streamId}	Download video files of a particular stream	▼
GET	/v1/storage/file/{streamId}/url	Get temporary URL for accessing stored video files	▼
GET	/v1/storage/{streamId}	Get the total space occupied by the video recordings of a stream	▼

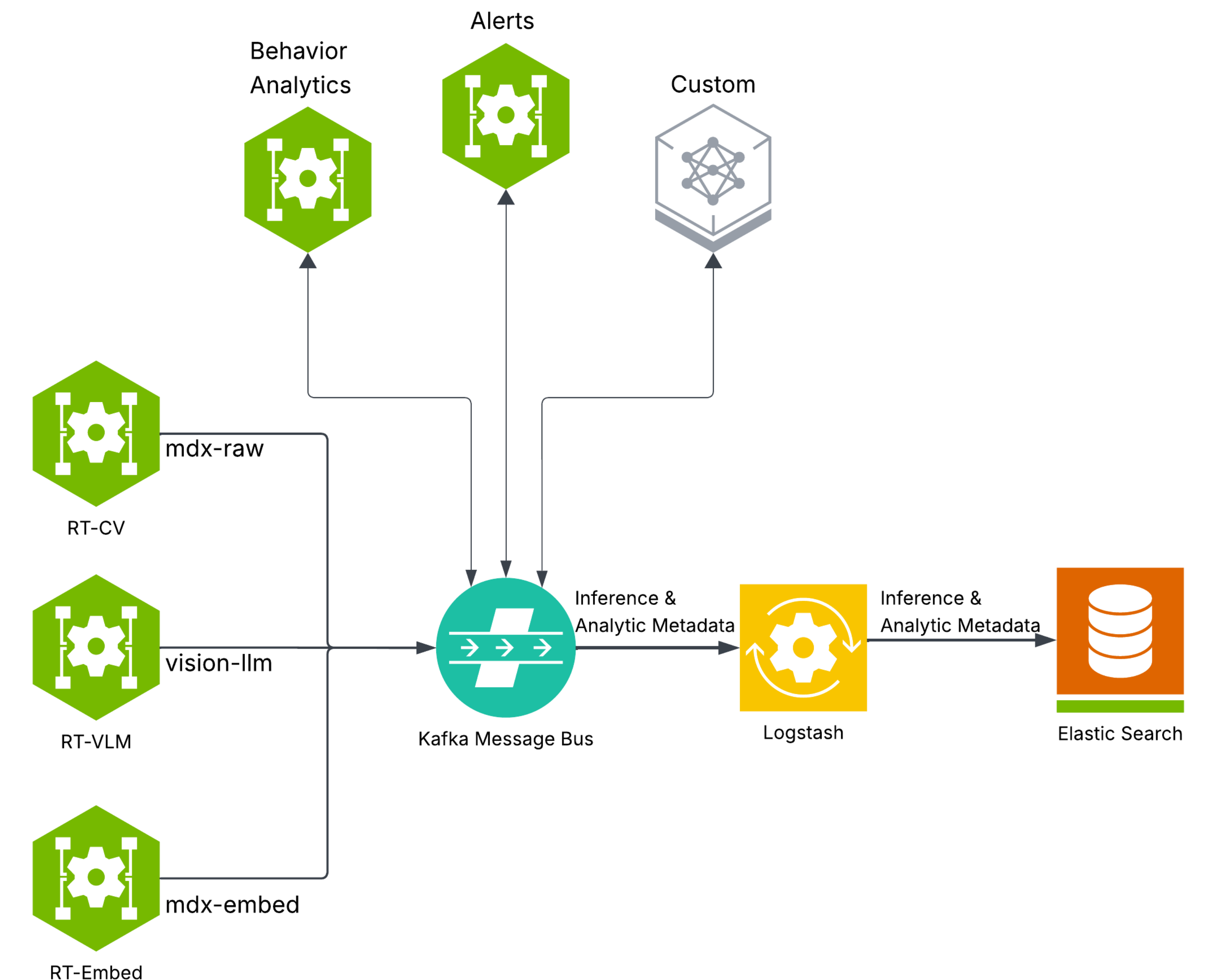
Kafka

Message Bus

- Connection between Vision Microservices, Analytics and Elastic Search
- Entry point to read/publish new information and implement new microservices
- Produce/Consumer message bus
- Information published on Topics
- Vision Microservices encode messages in Protobuf for efficient data transport

References

- [Kafka — VSS](#)
- [NvSchema — VSS](#)
- [Apache Kafka](#)



```
16:10:30.652 mdx-events Camera_02 Forklift #355003 ROI ENTRY ▶ Loading Area
16:10:30.752 mdx-events Camera_01 Forklift #355008 ROI ENTRY ▶ Loading Area
16:10:30.952 mdx-events Camera Forklift #355007 ROI ENTRY ▶ Loading Area
16:10:32.352 mdx-events Camera_01 Person #355016 ROI ENTRY ▶ Loading Area
16:10:32.385 mdx-events Camera Person #355015 ROI ENTRY ▶ Loading Area
16:10:31.719 mdx-events Camera_02 Person #355013 ROI ENTRY ▶ Loading Area
16:10:35.152 mdx-events Camera Forklift #355007 TRIPWIRE OUT ▶ Trailer Opening
16:10:35.152 mdx-events Camera Forklift #355007 ROI EXIT ▶ Loading Area
16:10:35.352 mdx-events Camera_01 Person #355009 ROI EXIT ▶ Loading Area
16:10:35.352 mdx-events Camera_01 Forklift #355008 TRIPWIRE OUT ▶ Trailer Opening
16:10:35.352 mdx-events Camera_01 Forklift #355008 ROI EXIT ▶ Loading Area
16:10:35.318 mdx-events Camera_02 Person #355005 ROI EXIT ▶ Loading Area
16:10:35.352 mdx-events Camera Person #355004 ROI EXIT ▶ Loading Area
16:10:35.419 mdx-events Camera_02 Forklift #355003 TRIPWIRE OUT ▶ Trailer Opening
16:10:35.419 mdx-events Camera_02 Forklift #355003 ROI EXIT ▶ Loading Area
```

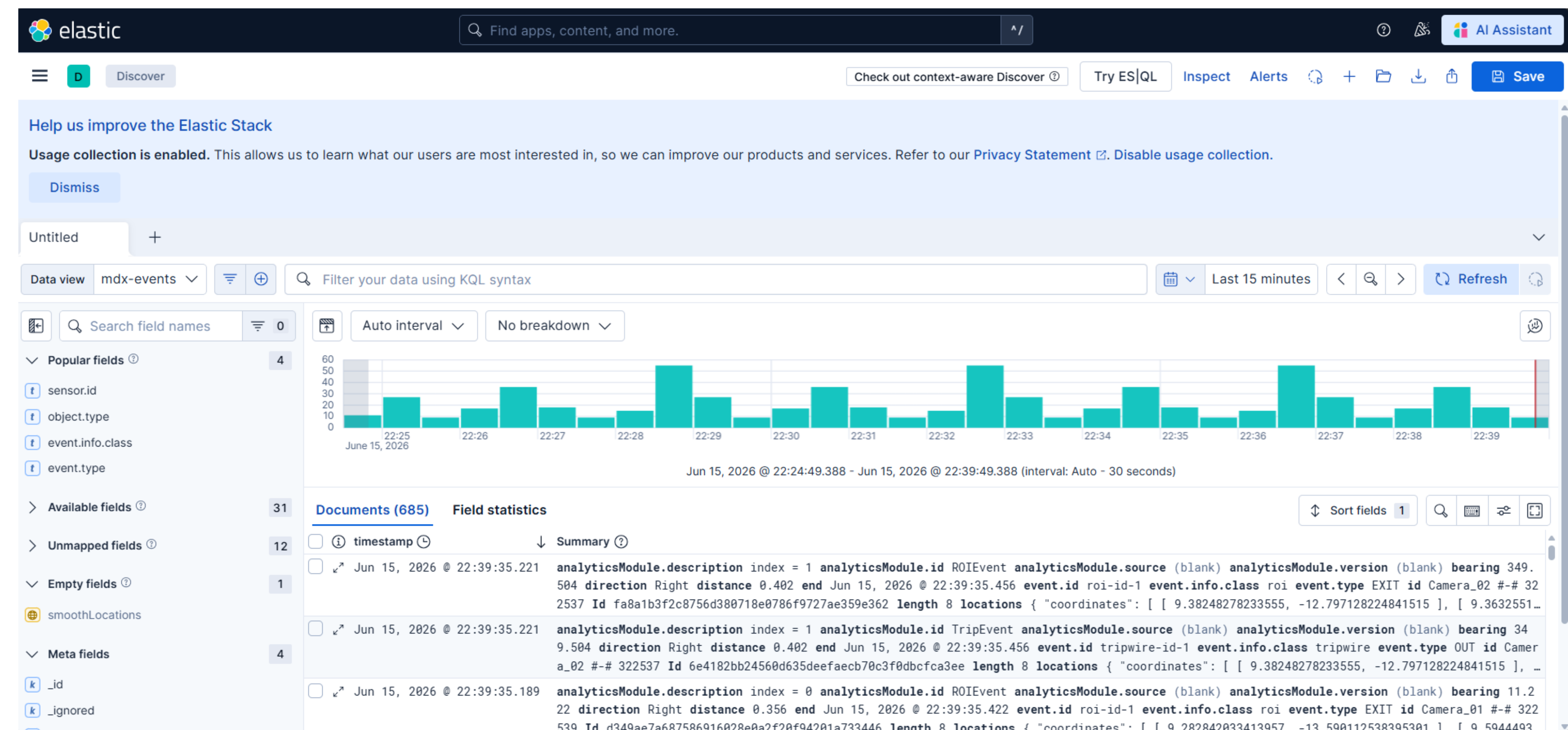
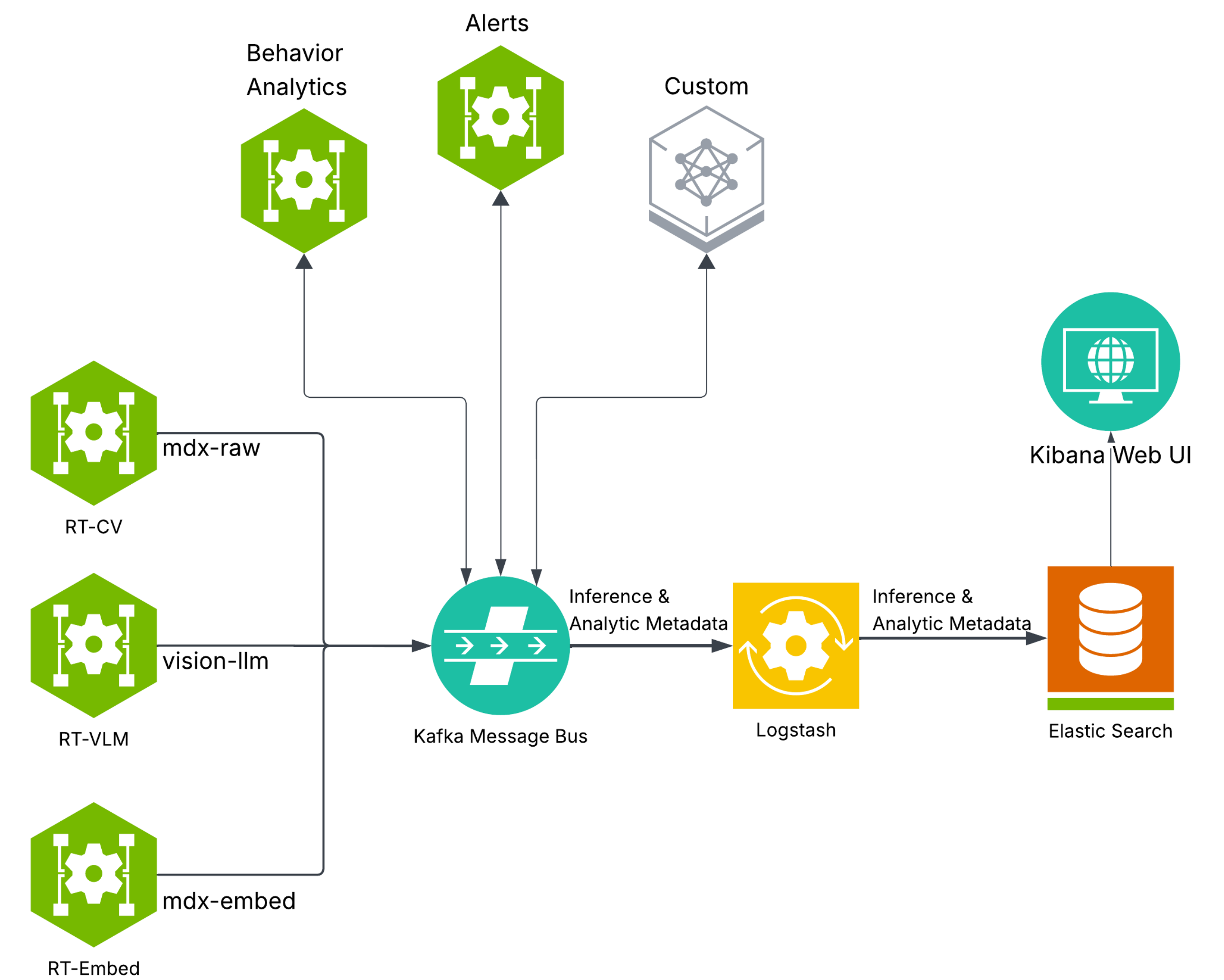
ELK

Elastic Search, Logstash, Kibana

- **Elastic Search** – Database to store timestamped video metadata and embeddings
- **Logstash** – Reads Kafka messages, and pushed into Elastic Search
- **Kibana** – Visualize all data stored in Elastic Search through Web UI
- Retrieve data from ES through REST API or various programming language client
- ES GPU accelerated Vector search with CuVS

References

- [ELK — VSS](#)
- [Elastic fundamentals | Elastic Docs](#)
- [Elasticsearch clients | Elastic Docs](#)



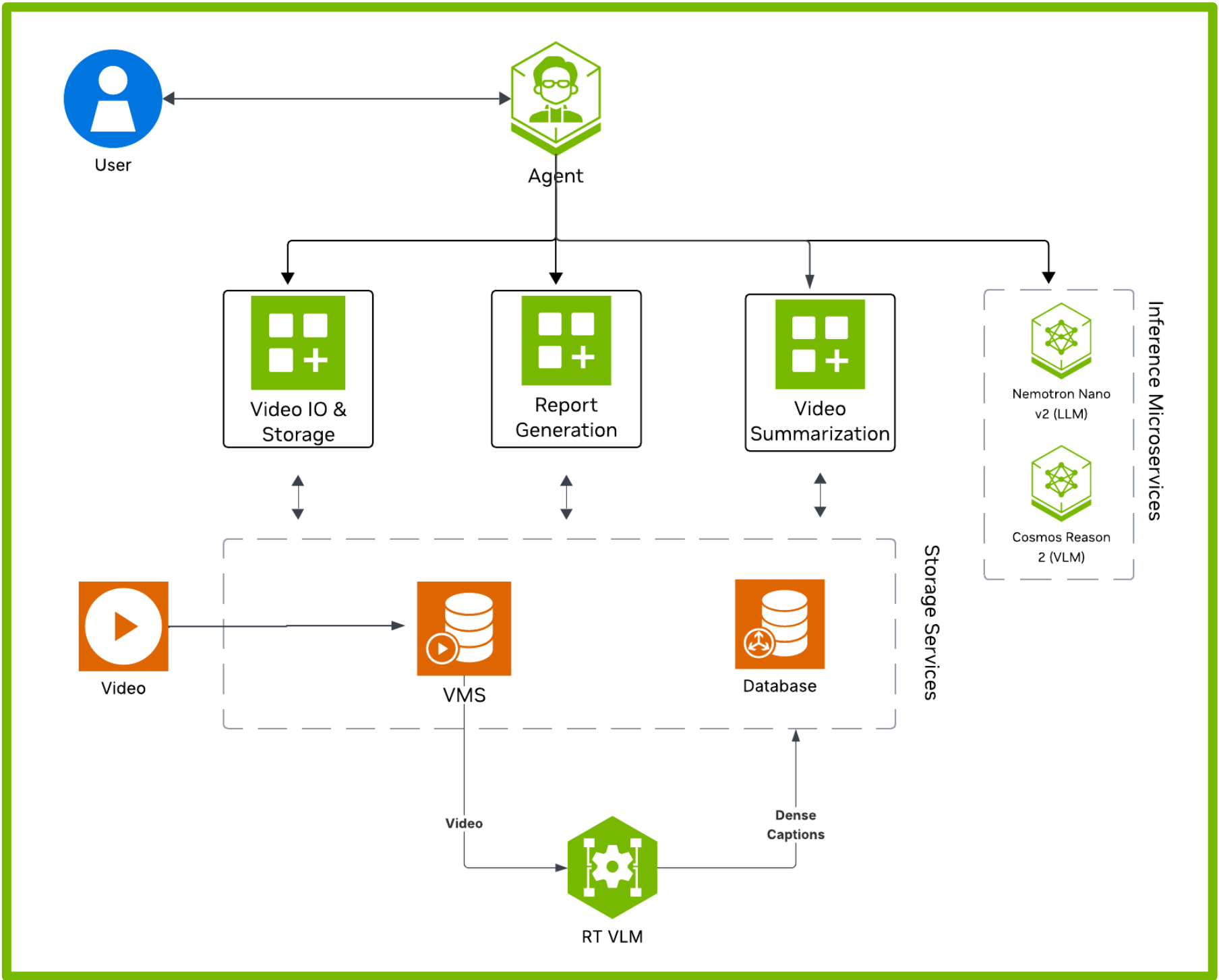
Agent Workflows



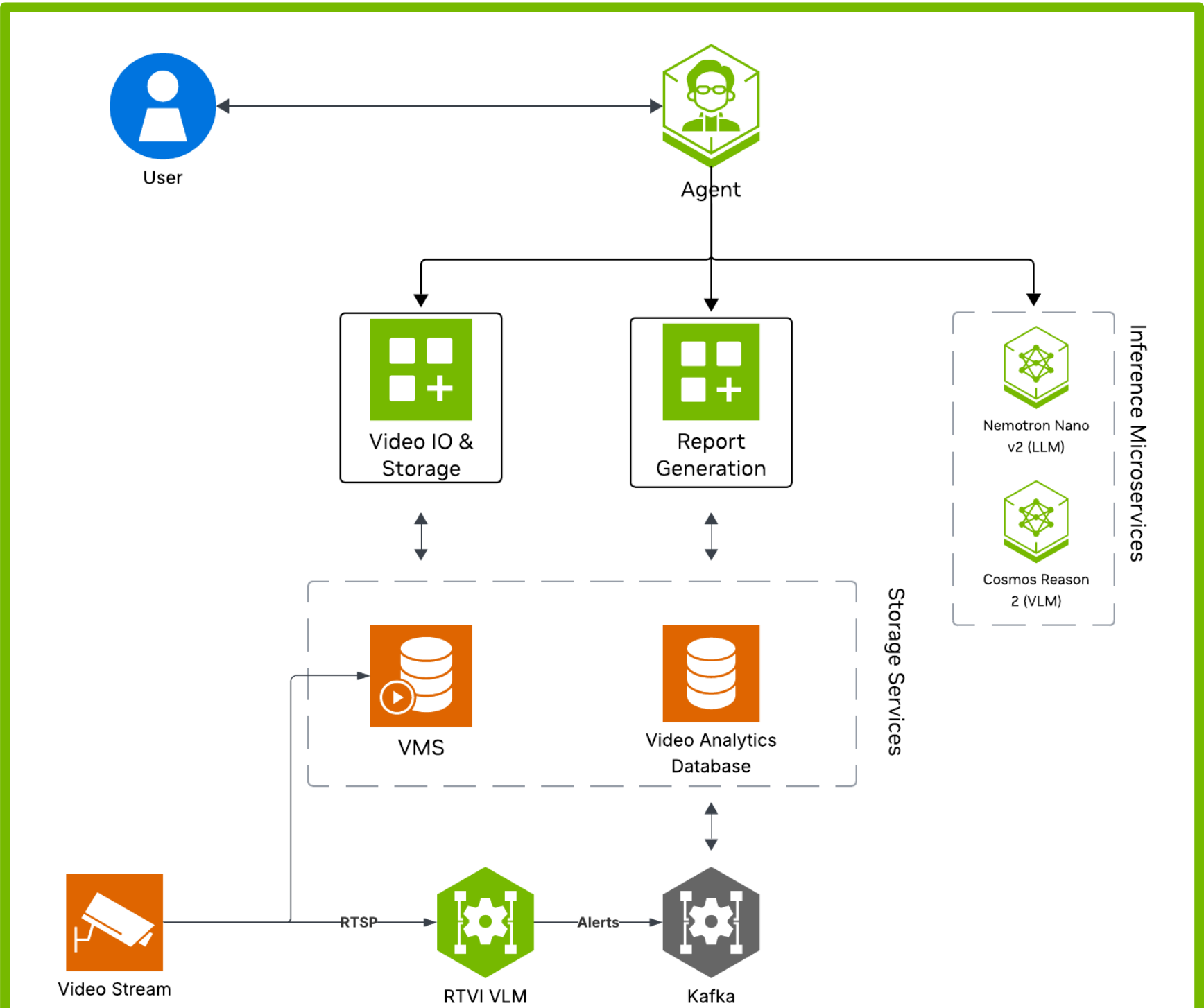
Agent Workflows

What are they?

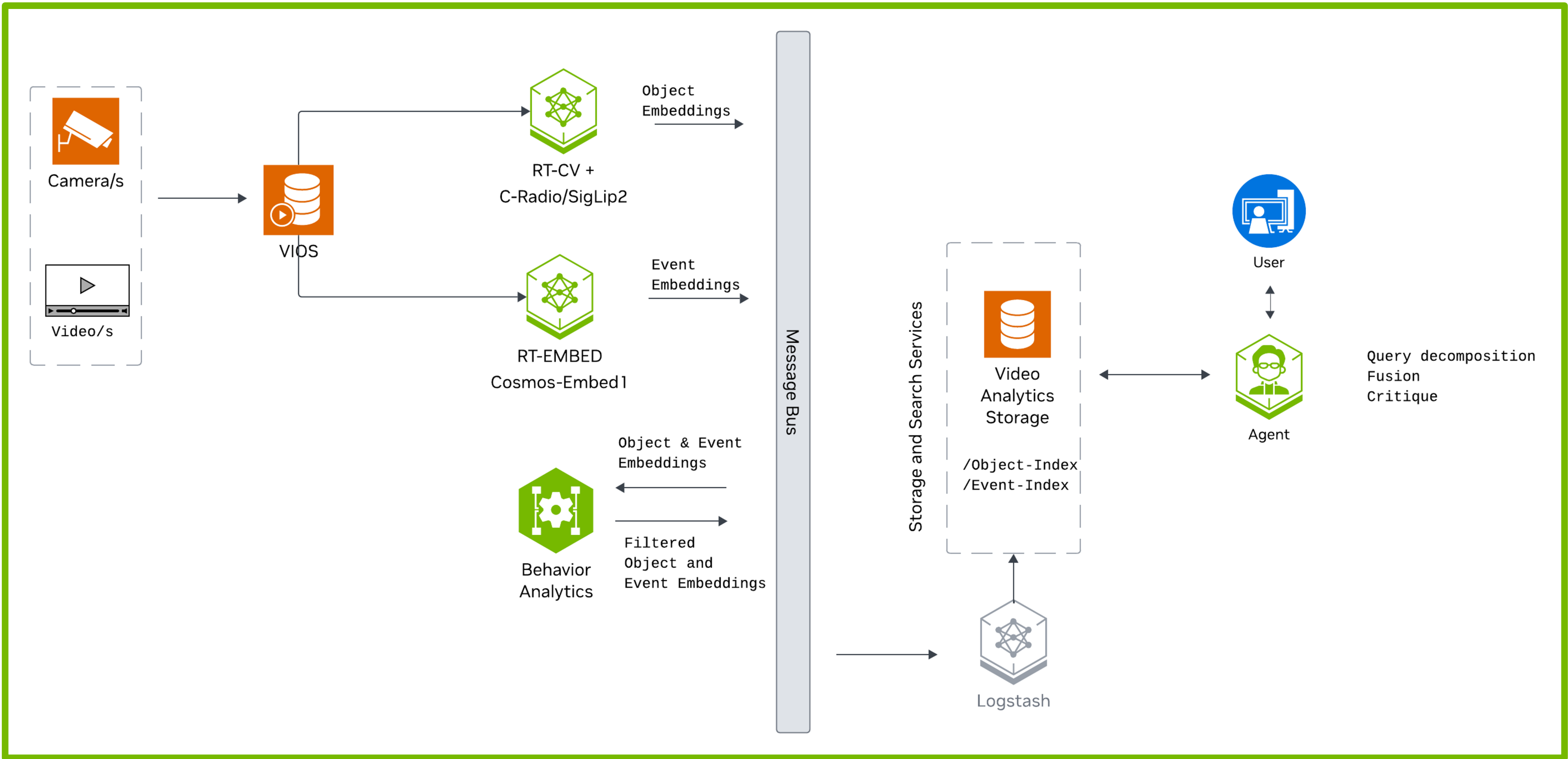
- VSS can be reconfigured to deploy task specific components
- Agent Workflows provide pre-built docker-compose based deployment for
 - Base – Video Q&A and Report Generation
 - Video Search
 - Video Summarization
 - Realtime Alerts
 - Alert Verification
- New profiles with specific and custom components can be built manually or with Agent Skills
- Once deployed, prebuilt-workflows can be interacted through reference UIs, Agent Skills or REST APIs



Summarization Workflow



Alerts Workflow



Search Workflow

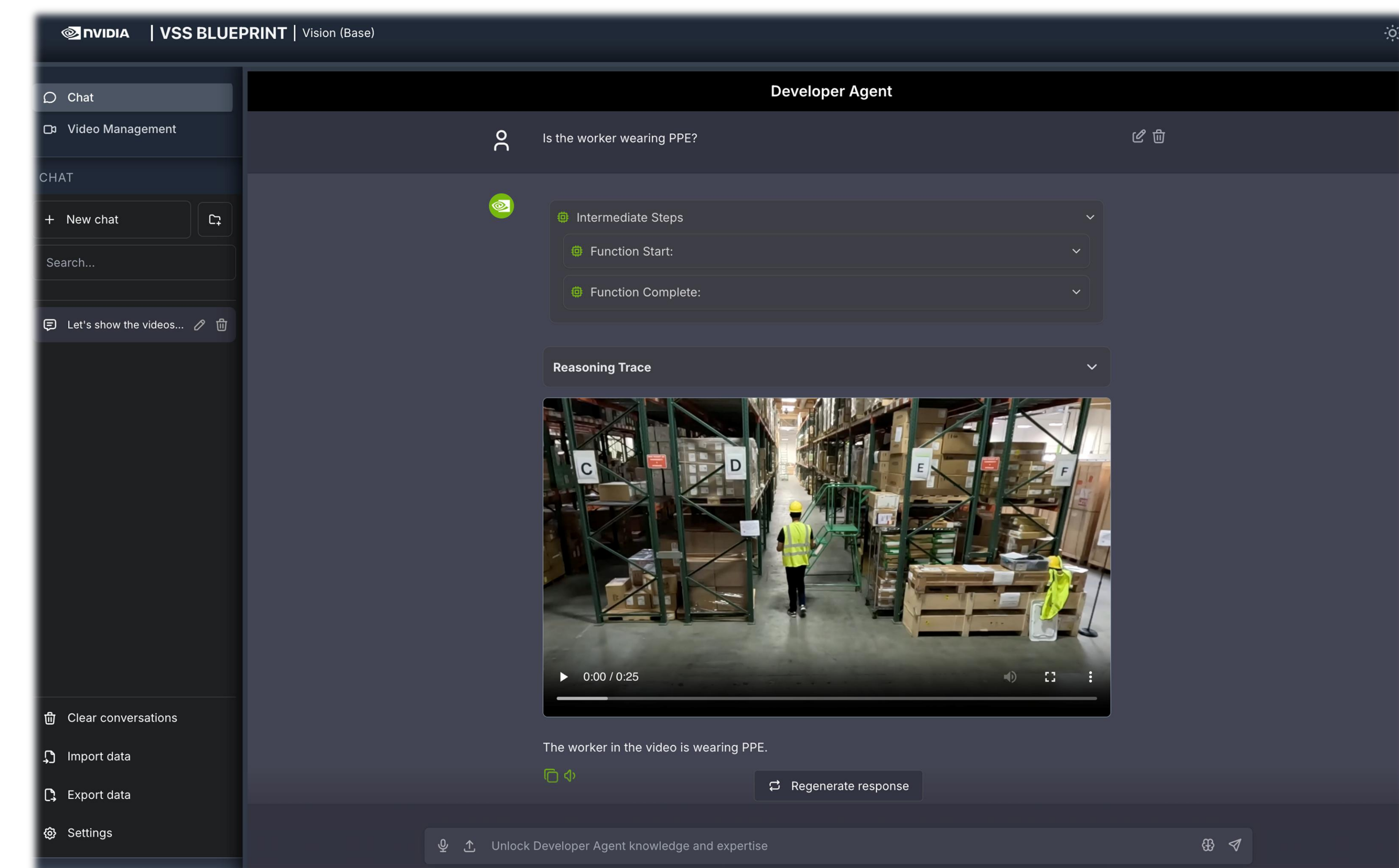
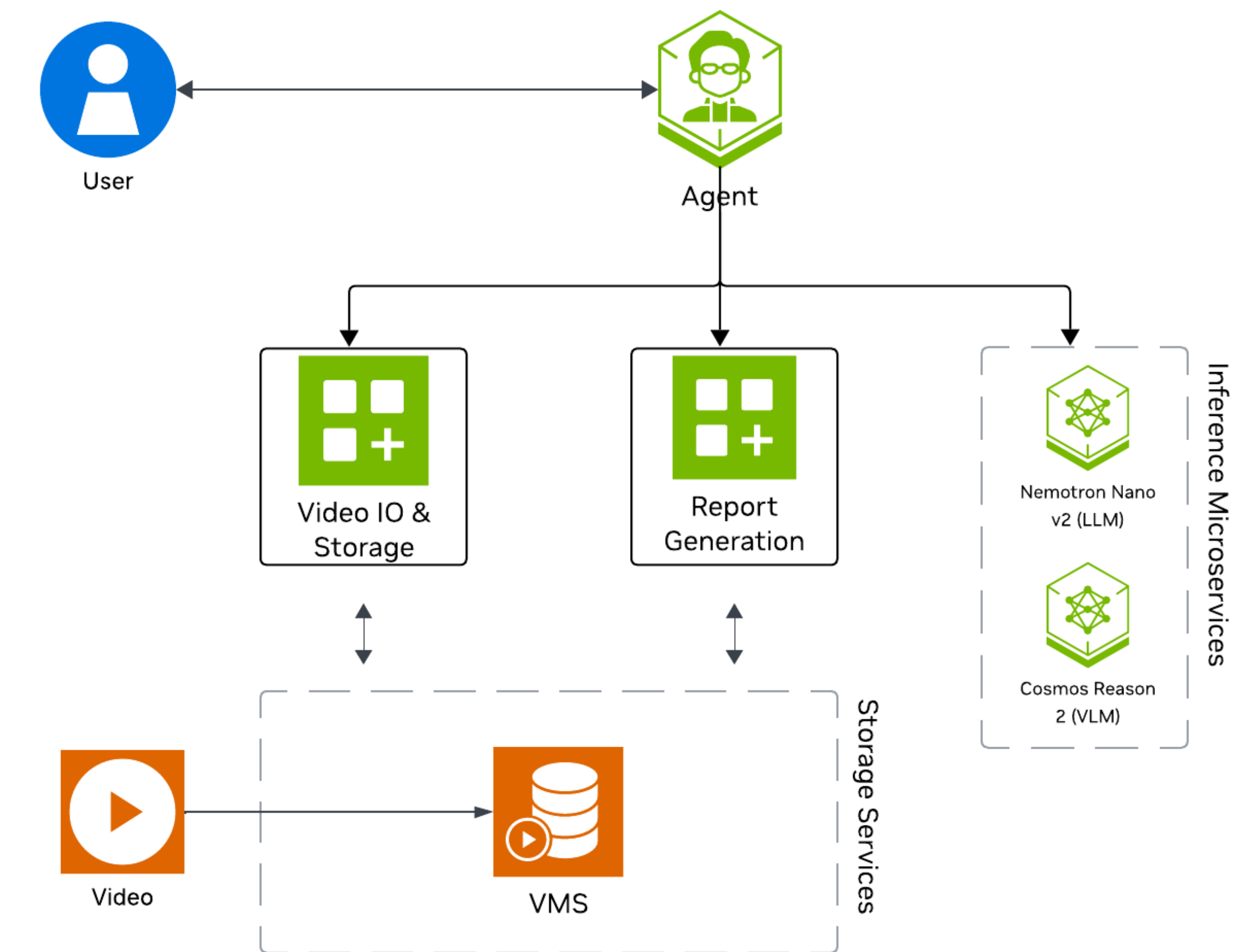
Base Profile

Usage

- VIOS: Video IO & Storage. Ingests streams/files, stores them, exposed over REST (:30888)
- Developer Agent: upload a video, ask in natural language, get reasoned answers
- Reports: auto-generated with timestamped observations
- Standalone: no search index, alerting, or analytics DB; the simplest profile to run
- Deploy with one docker compose profile (dev-profile-base); single-node, minimal GPU

References

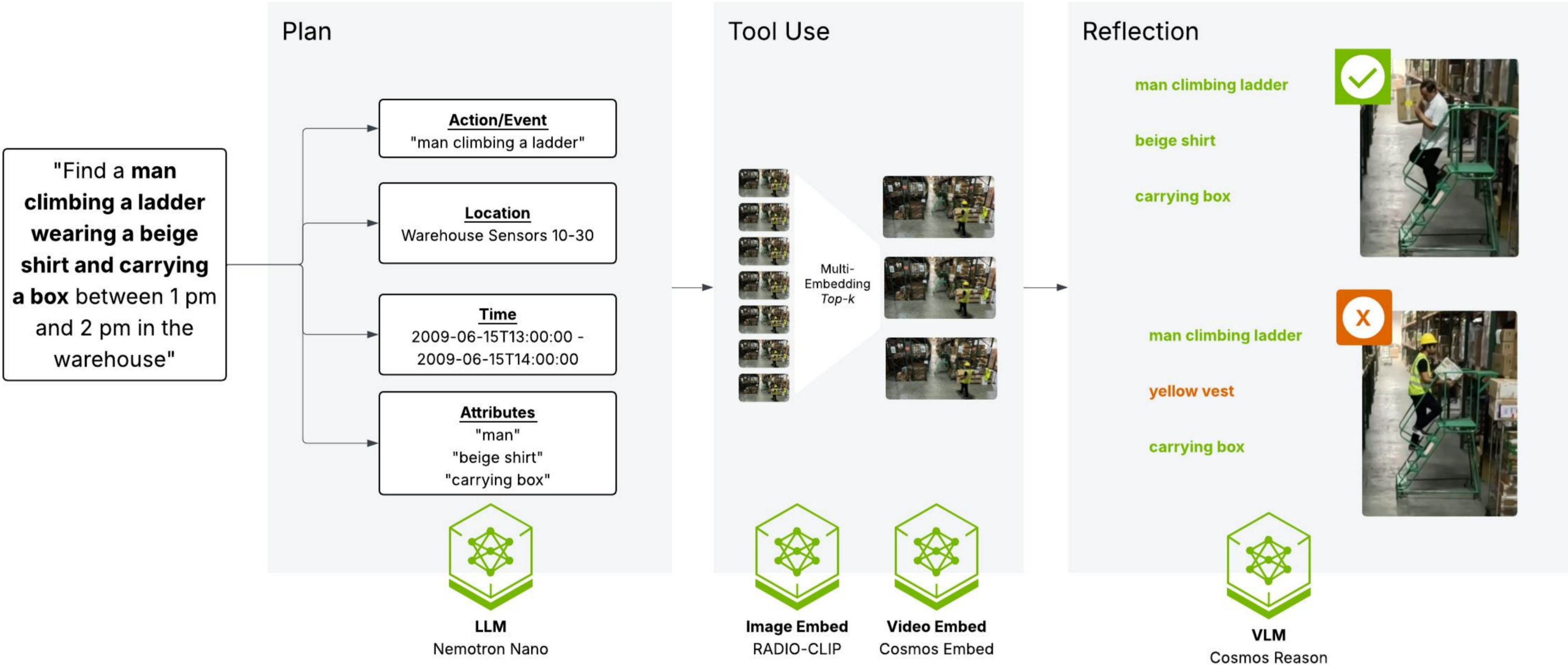
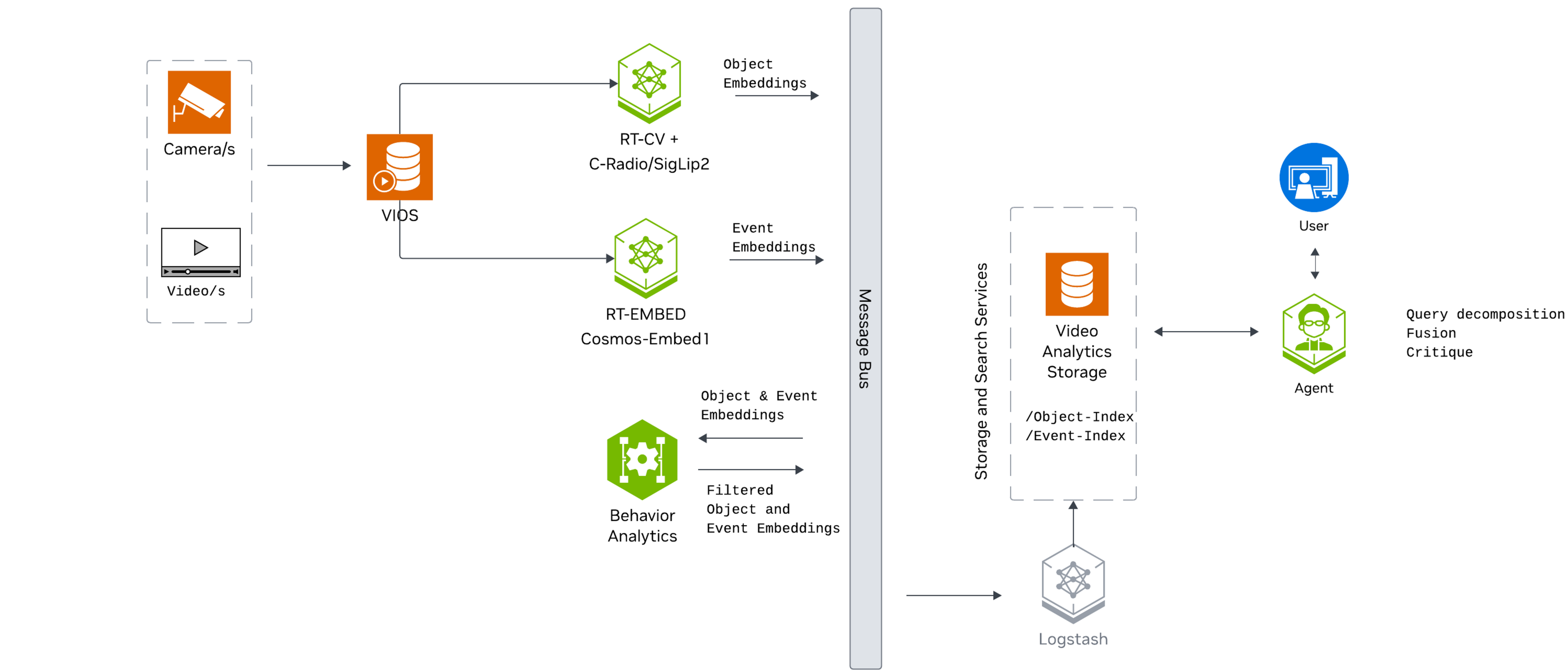
- [RT-VLM Performance — VSS](#)



Video Search Profile

Usage

- Event retrieval from large video archives
- Cross-video search for specific objects or actions
- Forensic analysis of recorded footage
- Deployed components
 - RT-CV & RT-Embed
 - VIOS & ELK
 - Behavior Analytics
 - Search Agent
- Optional VLM for verification



References

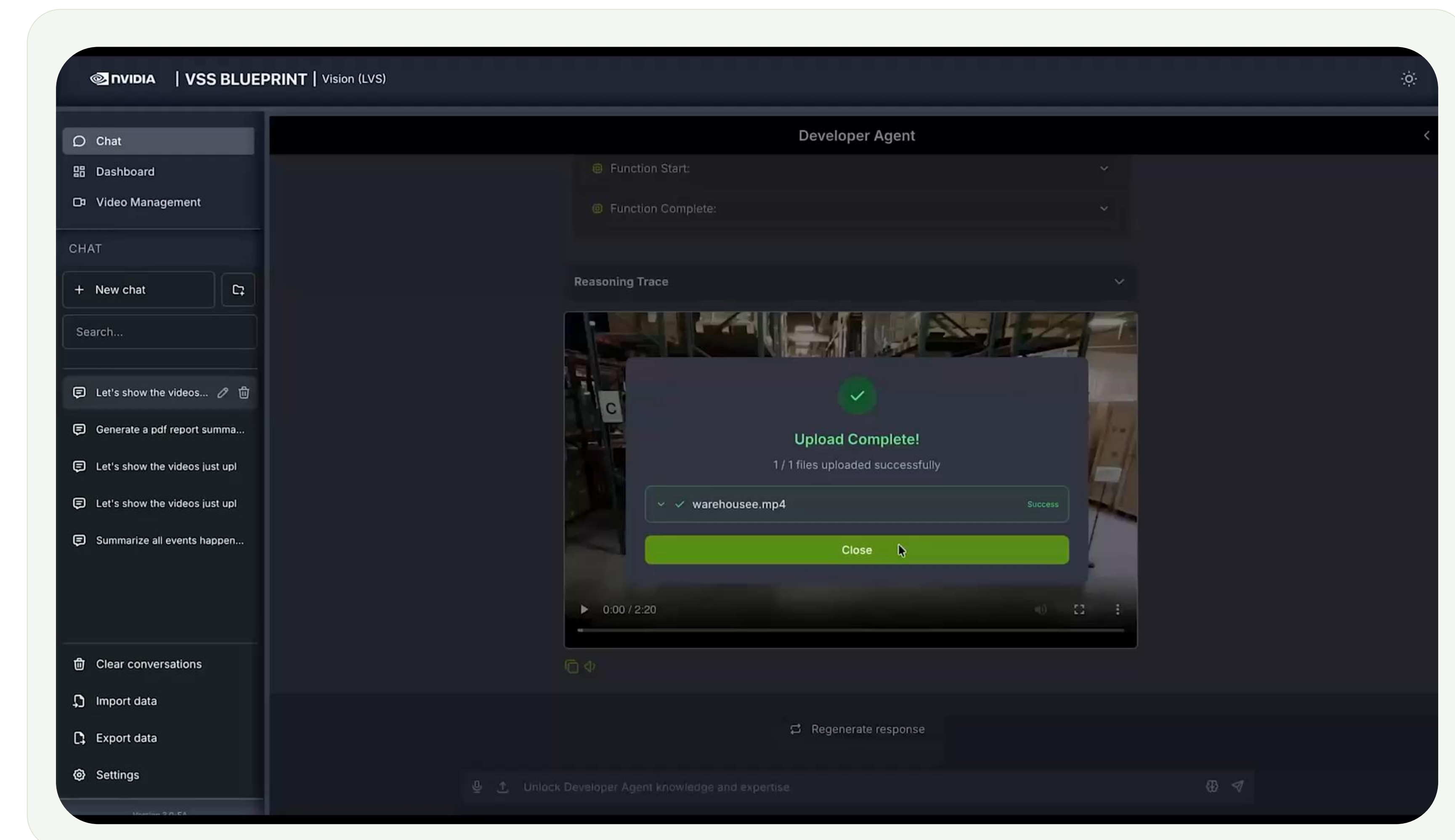
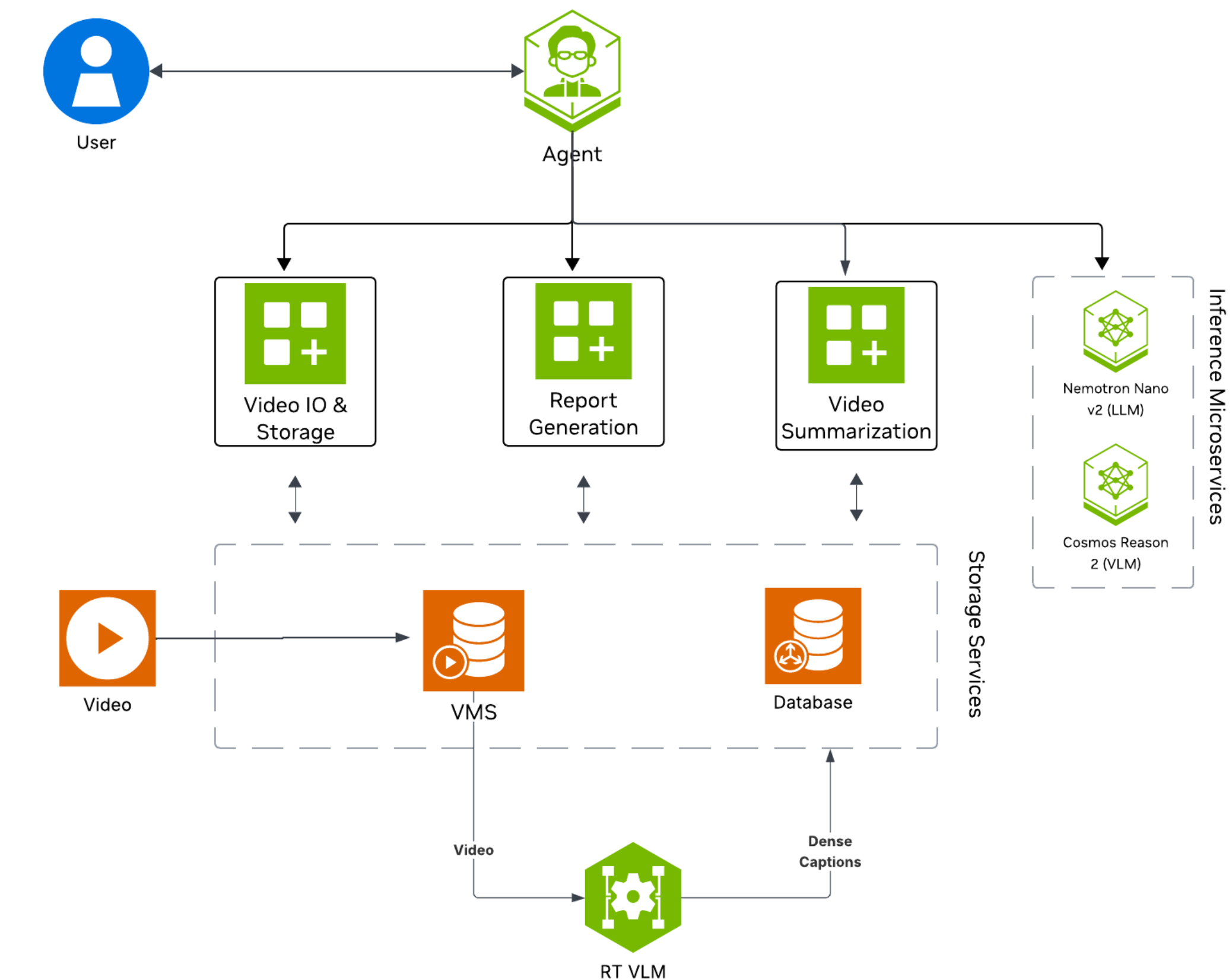
- [RT-VLM Performance — VSS](#)

Video Summarization Profile

- Dense captioning: Cosmos Reason VLM captions the video chunk by chunk
- Summarization: Nemotron LLM aggregates the captions into a structured summary
- HITL templates: you define the scenario and the events to watch for
- Report: timestamped events with embedded evidence frames
- File mode runs in-process; streaming mode adds Kafka for live RTSP summarization

References

- [Video Summarization Workflow — VSS](#)



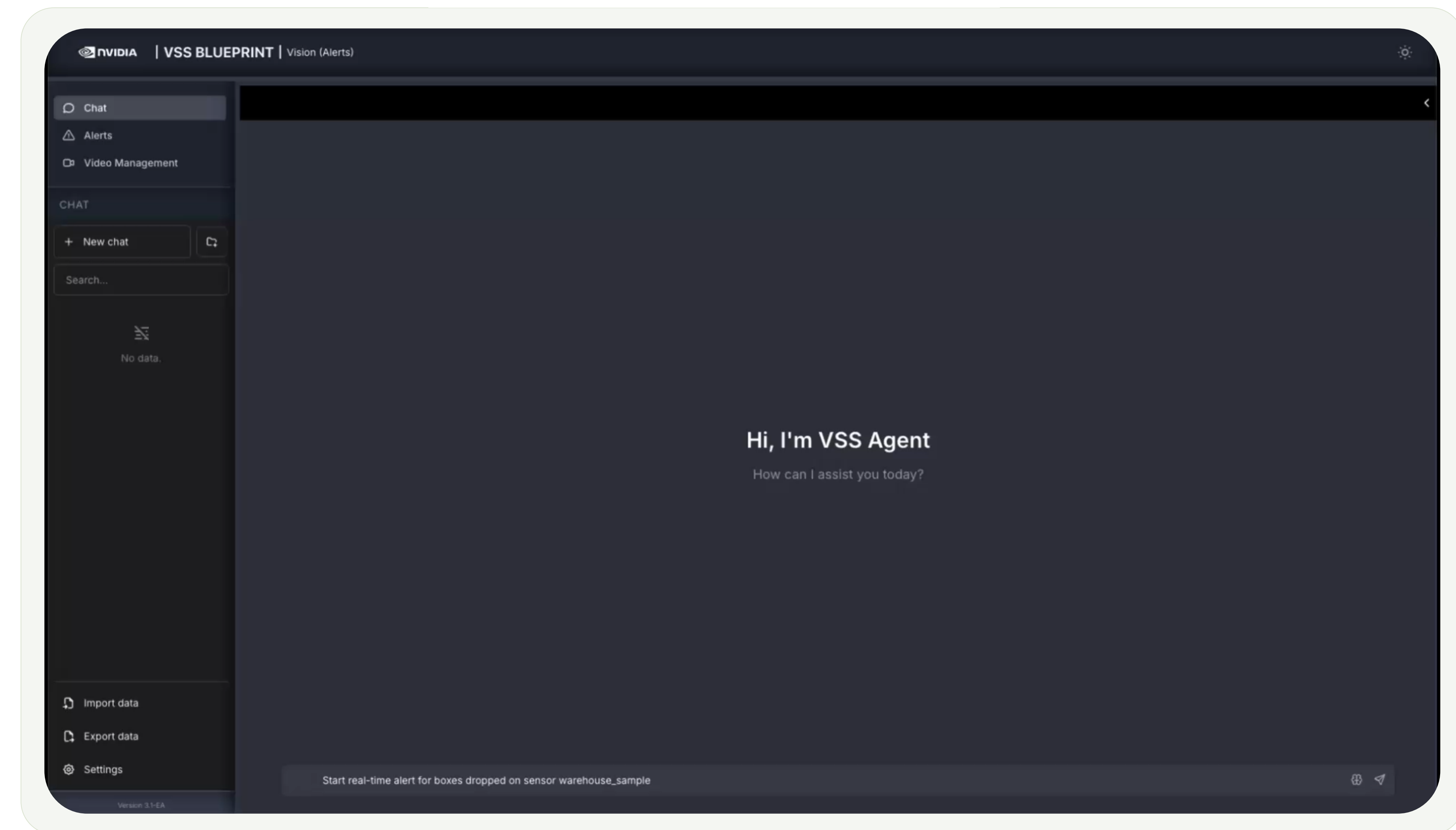
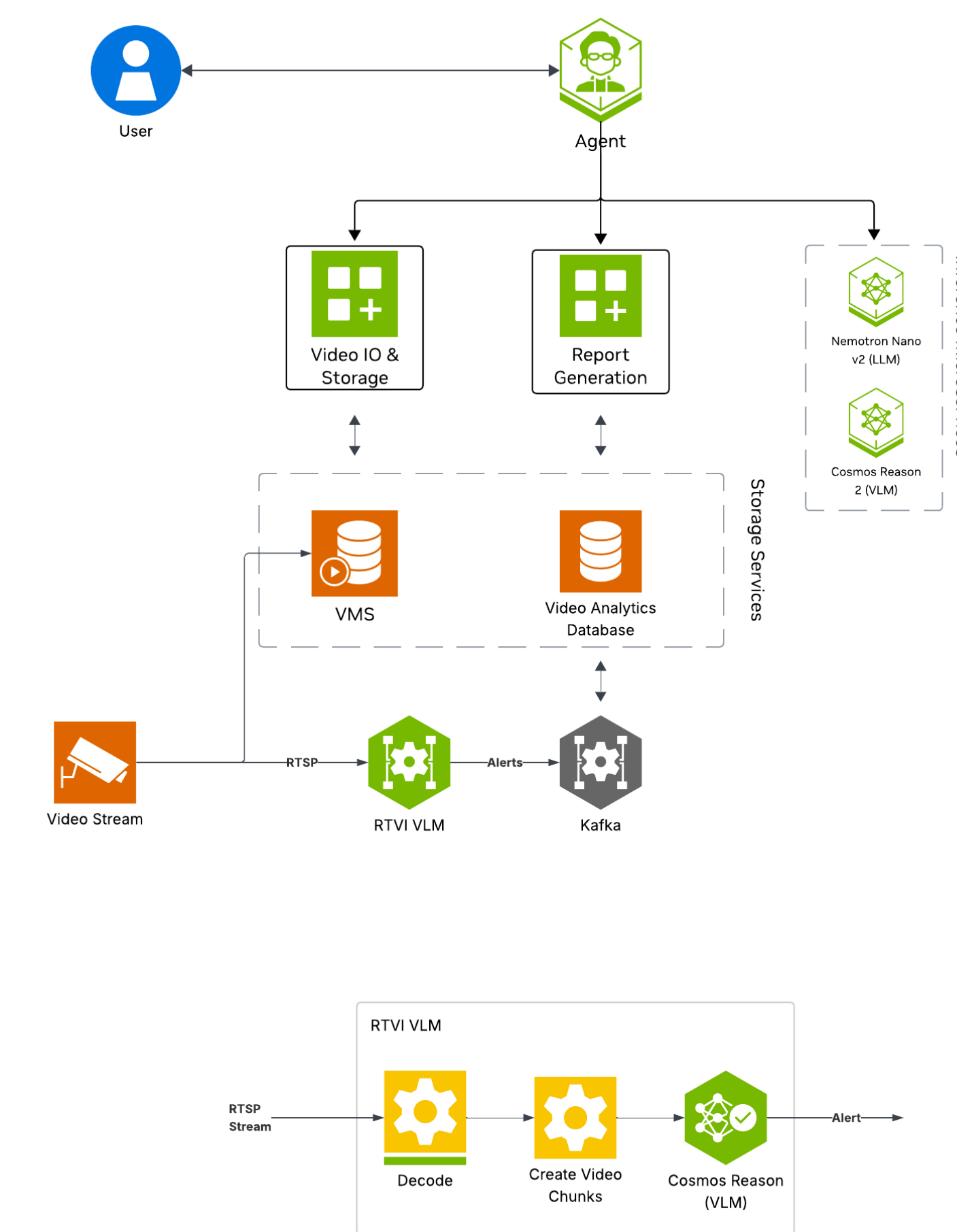
Real Time Alerts Profile

Usage

- RTVI-CV: RT-DETR (ResNet-50) detects objects on the live stream
- Behavior Analytics: watches for the alert condition you configured
- Cosmos Reason: wraps each trigger in a natural-language description
- Kafka: carries events after RTVI-CV, keeps end-to-end latency sub-second
- Contextual alerts a human can act on, not just "object in zone"

References

- [RT-VLM Performance — VSS](#)



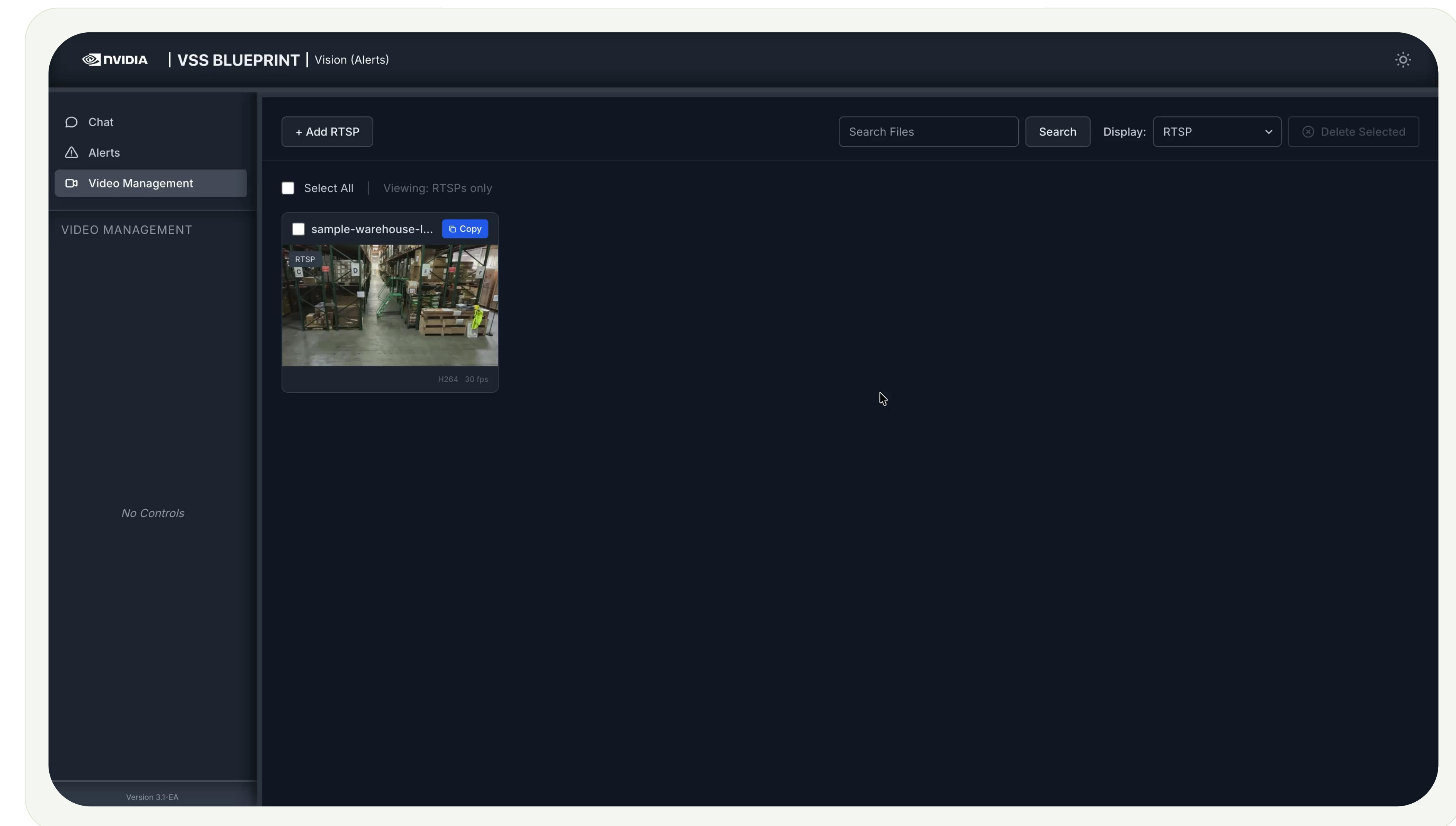
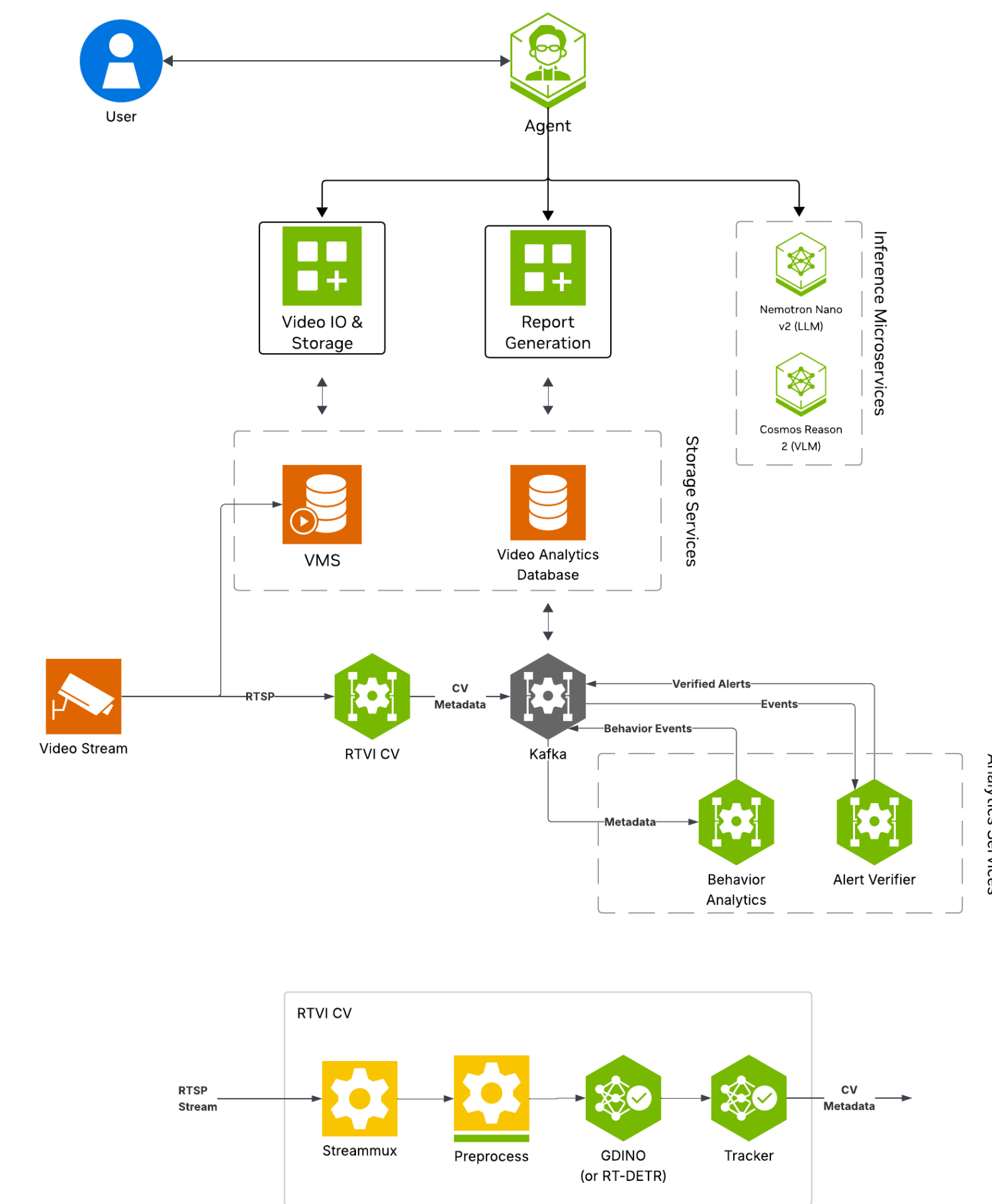
Alert Verification Profile

Usage

- Candidate alerts: RT-CV raises unverified detections
- Alert Verifier: Cosmos Reason VLM confirms or rejects each one
- Verified alerts: delivered with a natural-language justification
- False-positive reduction: the VLM gate is what makes alerts trustworthy
- A profile you enable, not a verification model you build yourself

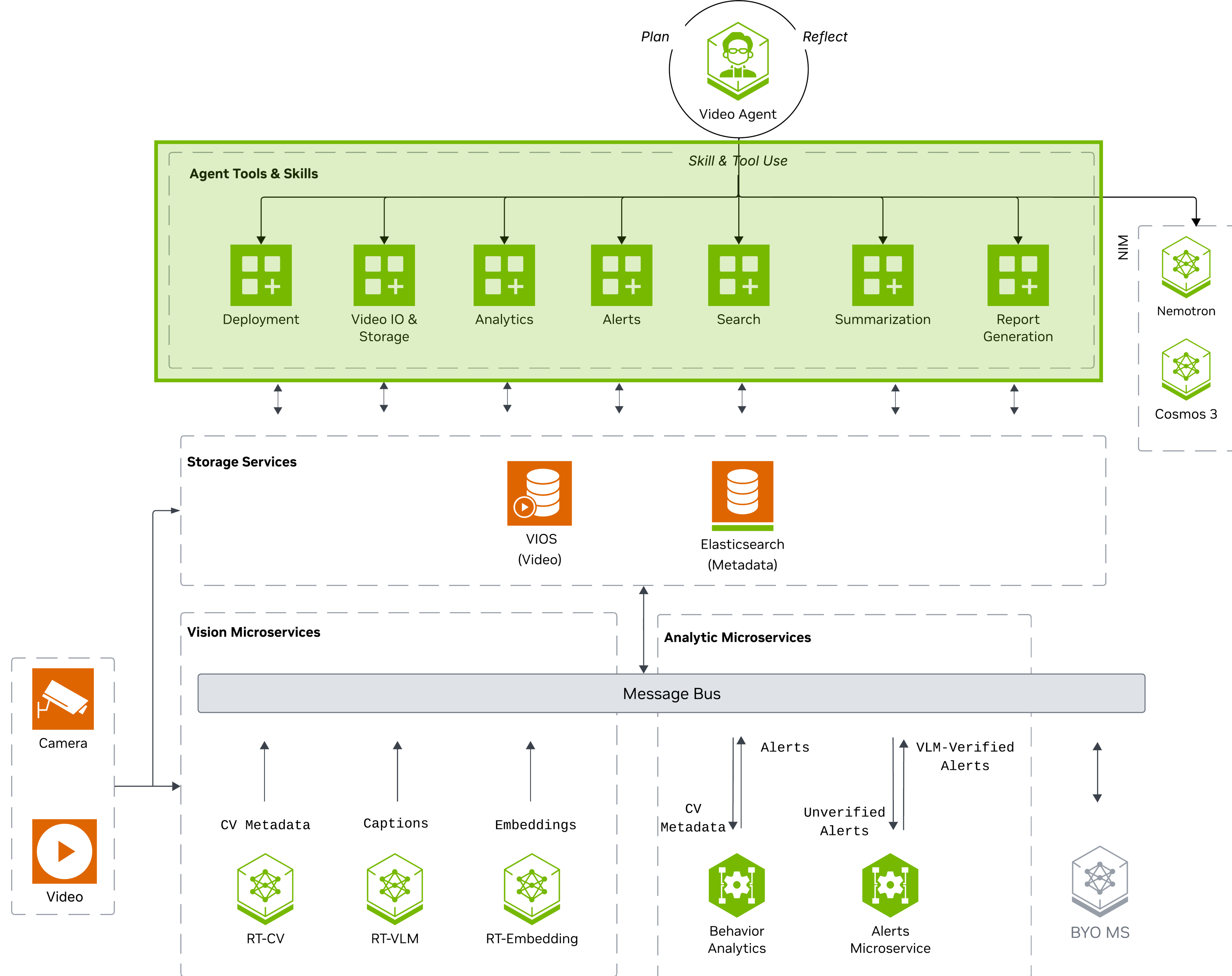
References

- [RT-VLM Performance — VSS](#)



Agent Tools & Skills

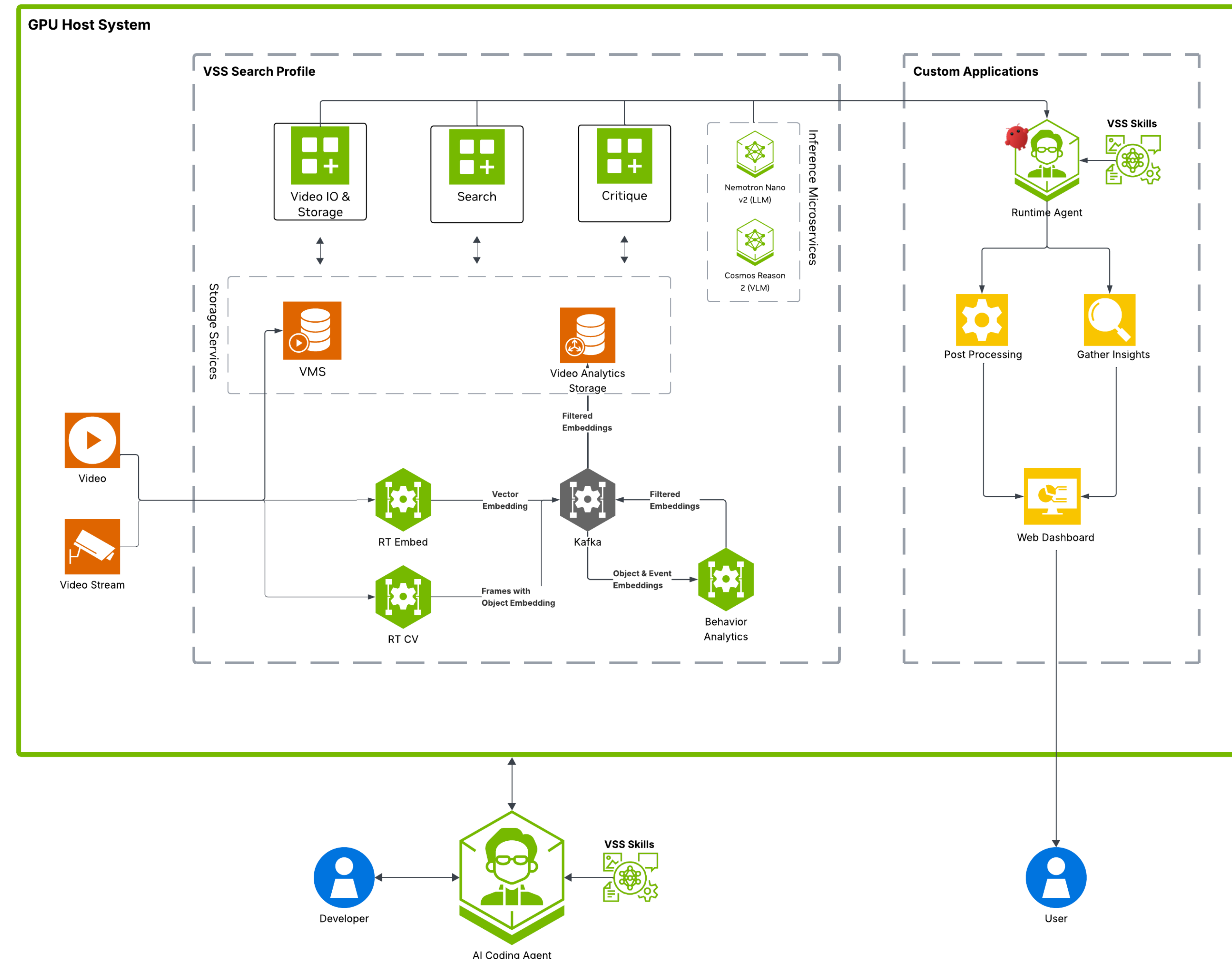




Agent Tools & Skills

What are they?

- VSS Agent Skills follow the agentskills.io spec
- Enhances AI agents with the ability to deploy, interact and build with VSS
- Can be used with nearly any coding or AI Agent
- Install skills by prompting your agent
 - “Install the VSS Skills from <https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization>
 - Or manually copy VSS Skills to your agent skills folder
- With VSS Skills, Agents understand the REST API Specs, Kafka and database schemas to interact with all layers of VSS



VSS Skills

<https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization/tree/main/skills>

Skills		Description
Deploy	vss-deploy-profile	Set up and manage VSS profiles with Docker Compose
	vss-deploy-dense-captioning	Run dense captioning microservice on stored video files and live RTSP streams
	vss-deploy-dense-tracking-2d	Deploy, debug, and operate the RTVI-CV detection / tracking 2D microservice
	vss-deploy-dense-tracking-3d	Deploy and operate the RTVI-CV-3D microservice as MV3DT
	vss-deploy-video-embedding	Deploy the VSS Video Embedding microservice from a Docker Compose file
Operate	vss-ask-video	Answer text questions about video content for frame-level VLM analysis
	vss-generate-video-calibration	Calibrate a multi-camera data using AMC from local MP4s, RTSP streams or sample dataset
	vss-generate-video-report	Generate analysis reports and format as per standard markdown report
	vss-generate-video-report-rag	Create summary reports using video_search_frag extension with RAG and HITL
	vss-manage-alerts	Add, manage, and monitor alerts on streamed video
	vss-manage-video-io-storage	Manage video and stream operations using the Video IO and Storage (VIOS) microservices
	vss-query-analytics	Query video analytics data and metrics from Elasticsearch via the VA-MCP server
	vss-search-archive	Search video archives using natural language with embeddings and CV attribute matching
	vss-setup-behavior-analytics	Set up behavior analytics service
	vss-setup-video-analytics-api	Deploy video analytics REST API with config, data-log, Elasticsearch, and optional Kafka wiring
	vss-summarize-video	Summarize videos through the Long Video Summarization microservice

VSS Skills

Usage

Install

Step 1: Clone the VSS Blueprint repository

```
git clone https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization.git
cd video-search-and-summarization
```

Step 2: Ask the coding agent to install the Skills

Read skills/README.md and every SKILL.md file under skills/. For each skill in the catalog, install it for this host so I can invoke it from a shell or chat session. Use the host's standard skills directory. Symlink each skill folder rather than copying it so a git pull here keeps every install up to date. Skip skills that are already installed and pointing at this checkout.

Use

Deploy VSS for video summarization.

Add an RTSP camera at rtsp://10.0.0.5:554/stream1.

Send me a PDF report every hour summarizing what occurred in the video.

Summary



VSS 3 Architecture

Framework for building Visual AI Agents

AI Models

- Detect, Track, Embed & Caption videos
- Finetune models in TAO

Vision Microservices

- Real time REST API controlled pipelines to extract metadata from video streams with AI models
- Build custom pipelines with DeepStream

Storage Services

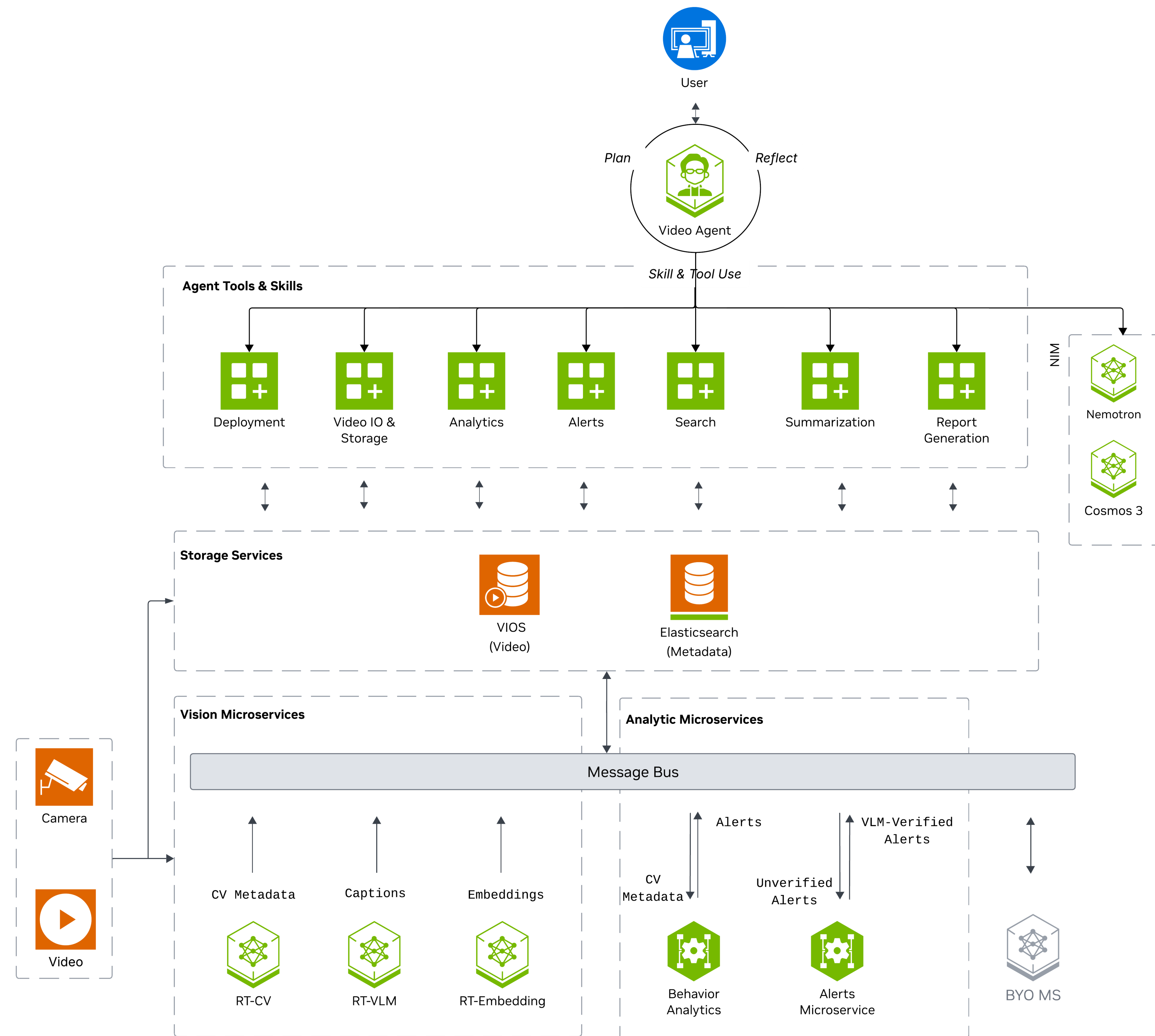
- VIOS – Video Database to manage, store, retrieve video streams and files
- Elastic Search – Database to store all extracted metadata, inference and analytic results

Analytic Microservices

- Operate on metadata in real time to extract further insights
- ROI/Tripwire Alerts, Multicamera Tracking, VLM Alerts

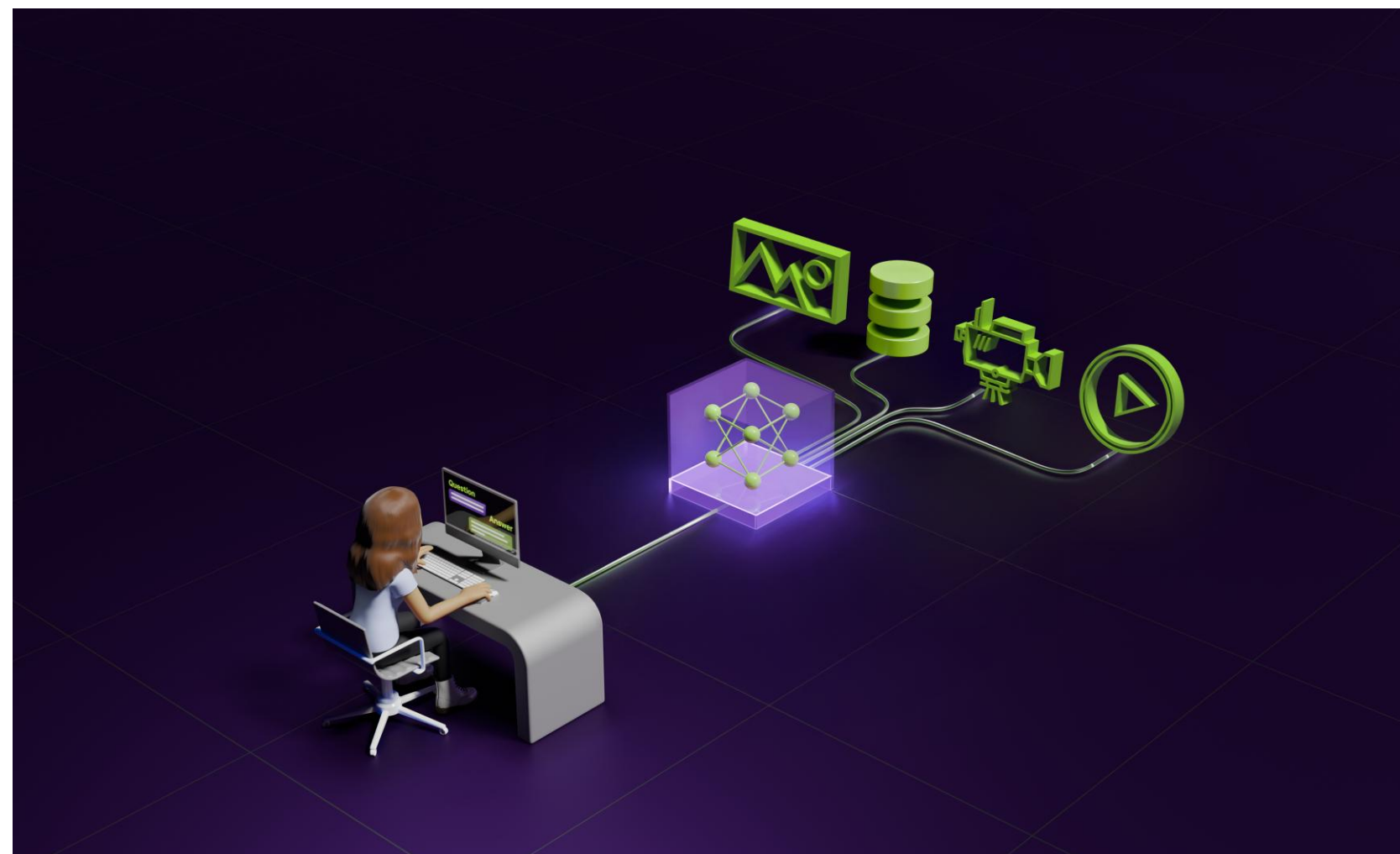
Agent Tools & Skills

- Pre-built tools for common usage and access patterns
- Skills for agentic use to deploy, build and interact with VSS



Resources

PRODUCT



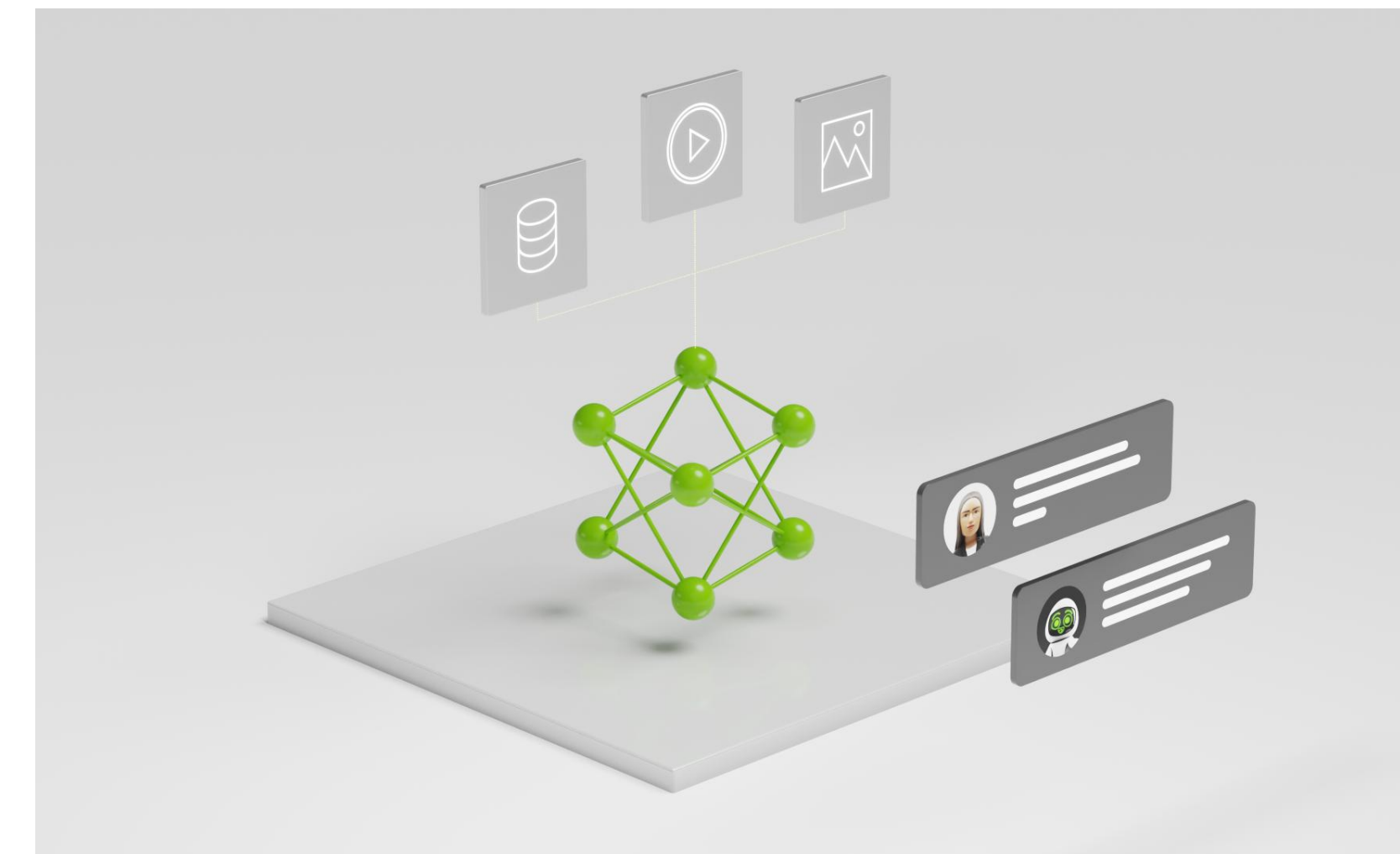
- VSS on build.nvidia.com
- [Product Deck](#)
- [VSS Messaging](#)
- [Developer Forum](#)

HOW TO GET STARTED



- [VSS Blueprint](#)
- [VSS Documentation](#)
- Try VSS on [Launchable](#)

INFORMATIONAL ASSETS



Blogs

- [VSS 3.2 Skills](#)
- [v2.3.0 Release](#)
- [VSS 101](#)
- [VLM Prompting](#)

Web Pages

- [AI Agents Use Case Page](#)
- [VLM Glossary Page](#)

Videos

- [Build Agent with NemoClaw](#)
- [VSS Use Cases Montage](#)
- [Fusion Search](#)

SALES COLLATERAL



Presentation Decks

- [Product Deck](#)

Partner Case Studies

- [Pegatron](#)
- [Linker Vision](#)

Livestream

- [Build AI Agents with Skills](#)

